

IOT-BASED HEALTH RISK PREDICTION SYSTEM USING MACHINE LEARNING WITH WEB ENABLED ALERTS

*A Project Report submitted in partial fulfillment of the requirement for the
award of the Degree of*

BACHELOR OF TECHNOLOGY

In

**ELECTRONICS AND COMMUNICATION
ENGINEERING**

By

**KANDRAPU NIKITHA
(322103212043)
DARRU BHAGYA SRI
(322103212014)**

**DUNNA USHASRI
(322103212023)
BALIVADA SWARNA LATHA
(322103212004)**



Under the Esteemed guidance of

MRS. CH. SIRISHA

Assistant Professor

**DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING
GAYATRI VIDYA PARISHAD COLLEGE OF ENGINEERING FOR WOMEN
(Affiliated to Andhra University, Visakhapatnam)
MADHURAWADA, VISAKHAPATNAM-48
(2022-2026)**

GAYATRI VIDYA PARISHAD COLLEGE OF ENGINEERING FOR WOMEN
DEPARTMENT OF ELECTRONICS AND COMMUNICATION ENGINEERING



CERTIFICATE

This is to certify that the project titled “**IOT-BASED HEALTH RISK PREDICTION SYSTEM USING MACHINE LEARNING WITH WEB ENABLED ALERTS**” is a bonafide work of the following 1V/1V B-Tech students in the Department of Electronics and Communication Engineering during the academic year 2025-26, in partial fulfillment of the requirement for the award of the degree of Bachelor of Technology of Andhra University, Visakhapatnam.

Kandrapu Nikitha (322103212043)

Dunna Ushasri (322103212023)

Darru Bhagya Sri (322103212014)

Balivada Swarna Latha (322103212004)

Project Guide

Mrs. Ch. Sirisha

Assistant Professor

Head of the Department

Dr. P.M.K.Prasad

Professor

External Examiner

ACKNOWLEDGEMENT

We sincerely thank our project guide **Mrs.Ch.Sirisha**, Assistant Professor, for her guidance and constant encouragement to us at every stage and aspect by including the spirit of understanding and support in carrying out our main project at college.

We would like to sincerely thank our project coordinator **Dr. M. Mani Kumari** Assistant professor for her insightful advice and encouragement, both of which were extremely helpful to me in successfully finishing the Project work.

We would like to express sincere thanks to our Head of the Department of Electronics and Communication Engineering **Dr.P.M.K.Prasad** for his valuable suggestions and constant motivation that greatly helped me in completing the Project work successfully.

We express sincere thanks to our Vice Principal, Professor **Dr.G.Sudheer**, for his encouragement and co-operation in completion of our project.

We wish to express our deep sense of our gratitude to our Principal, Professor **Dr.R.KGoswami**, for giving us the opportunity to carry out the project work successfully.

We would like to express our gratitude towards our parents & members of Gayatri Vidya Parishad College of Engineering for Women for their kind co-operation and encouragement which helped us in completion of our project.

Kandrapu Nikitha (322103212043)

Dunna Ushasri (322103212023)

Darru Bhagya Sri(322103212014)

Balivada Swarna Latha (322103212004)

VISION AND MISSION

Vision of the Institute

To emerge as an acclaimed center of learning that provides value-based technical education for the holistic development of students

Mission of the Institute

- Undertake the activities that provide value-based knowledge in Science, Engineering, and Technology
- Provide opportunities for learning through industry-institute interaction on the state-of-the-art technologies
- Create a collaborative environment for research, innovation, and entrepreneurship
- Promote activities that bring in a Sense of social responsibility

Vision of the Department

Produce competitive engineers instilled with ethical and social responsibilities to deal with the technological challenges in the field of electronics & Communication Engineering.

Mission of the Department

- Facilitate a value-based educational environment that provides updated technical knowledge
- Provide opportunities for developing creative, innovative and leadership skills
- Imbue technological and managerial capabilities for a successful career and lifelong learning

ABSTRACT

A predictive healthcare framework is developed by integrating IoT-enabled sensors with Machine Learning for real-time risk assessment. Patient vital parameters such as temperature, SpO₂ (from a pulse oximeter), and heart rate, blood pressure are collected using an Arduino-based system. The acquired data is transmitted to a local edge environment, where a Flask-based REST API processes the inputs using a Logistic Regression model to predict the probability of the patient's risk level. The patient's real-time location is obtained through the Browser Geolocation API, and nearby hospitals are identified using the Overpass API from OpenStreetMap. When a critical condition is detected, the system automatically generates a high-risk alert and sends it to the nearest hospital's web portal, requesting patient admission and ambulance dispatch. This framework enhances remote patient monitoring, reduces preventable hospital readmissions, and supports proactive healthcare delivery. It prioritizes high-risk patients for admission, improves hospital resource utilization, and ensures timely medical care.

CONTENTS

TITLE	PAGE NO
Certificate	ii
Acknowledgment	iii
Vision and Mission	iv
Abstract	v
List of Figures	x
CHAPTER-1 INTRODUCTION	1
1.1 Introduction to Health Prediction System	1
1.2 Aim	2
1.3 Motivation	2
1.4 Problem Statement	2
1.5 Objective	2
1.6 Applications	3
1.7 Organization of Thesis	3
CHAPTER-2 LITERATURE SURVEY	4
2.1 Reference Paper – 1	4
2.2 Reference Paper – 2	5
2.3 Reference Paper – 3	6
2.4 Conclusion on Survey Paper	7

CHAPTER-3 SYSTEM DESIGN AND METHODOLOGY	8
3.1 Proposed System	8
3.2 Block Diagram	8
3.3 Flowchart and Methodology	9
3.4 Conclusion	10
CHAPTER-4 HARDWARE IMPLEMENTATION	11
4.1 Components and Tools Required	11
4.1.1 Arduino UNO	11
4.1.2 Temperature Sensor	12
4.1.3 Pulse Oximeter	12
4.1.4 Blood Pressure Sensor	13
4.1.5 Arduino IDE	14
4.2 Circuit Implementation	15
4.3 Hardware Output	15
4.4 Conclusion	16
CHAPTER-5 SOFTWARE IMPLEMENTATION	17
5.1 Software Requirements	17
5.1.1 Programming Environment	17
5.1.2 Backend Framework	17
5.1.3 Serial Communication	18
5.1.4 Web Interface	18

5.1.5 Location Services	18
5.2 Machine Learning Model Development	18
5.2.1 Introduction	18
5.2.2 Flowchart	18
5.2.3 Dataset Collection	19
5.2.4 Data Preprocessing	20
5.2.5 Model Selection	20
5.2.6 Model Training	21
5.2.7 Model Evaluation & Performance	22
5.3 Flask Backend & Web Application	23
5.3.1 Introduction to Flask	23
5.3.2 Flask Application Setup	23
5.3.3 Data Transmission from Arduino to Flask	24
5.3.4 Model Deployment into Flask	25
5.3.5 Patient Web Application	26
5.3.6 Hospital Web Application	27
5.3.7 Flask Server Integration	29
5.3.8 Conclusion	31
CHAPTER-6 RESULTS AND ANALYSIS	32
6.1 Logistic Regression Model Results	32
6.1.1 Heart Rate Model	32

6.1.2 SpO ₂ Model	34
6.1.3 Blood Pressure Model	35
6.1.4 Temperature Model	36
6.1.5 Performance Summary of Models	37
6.2 System Output	37
6.2.1 No Alert Condition	38
6.2.2 Medium Alert Condition	39
6.2.3 High Alert Condition	40
6.3 Final Model	41
6.4 Conclusion	43
6.5 Future Scope	44
6.6 References	45

LIST OF FIGURES

FIGURE NO	TITLE	PAGE NO
Fig.1.1	IoT-Based Health Risk Prediction System Using ML With Web Enabled Alerts	1
Fig.3.1	Block Diagram	8
Fig.3.2	Flowchart	9
Fig.4.1	Arduino Uno	11
Fig.4.2	Temperature Sensor	12
Fig.4.3	Pulse Oximeter Sensor	13
Fig.4.4	Blood Pressure Sensor	13
Fig.4.5	Arduino IDE	14
Fig.4.6	Circuit Implementation	15
Fig.4.7	Hardware Output	16
Fig.5.1	Flowchart (Model Development)	19
Fig.5.2	Dataset	20
Fig.5.3	Model Training in Google Colab	22
Fig.5.4	Flask Application Setup	24
Fig.5.5	Data Transmission to Flask	25
Fig.5.6	Models Deployment into Flask	25
Fig.5.7	Login Page and Registration	26
Fig.5.8	Patient Dashboard	27

Fig.5.9	Data Storage	27
Fig.5.10	Hospital Login Page and Registration	28
Fig.5.11	Hospital Dashboard	28
Fig.5.12	Patient Risk Analysis	30
Fig.6.1	Heart Rate Alert Prediction	33
Fig.6.2	Heart Rate Classification Report and Confusion Matrix	33
Fig.6.3	SpO ₂ Alert Prediction	34
Fig.6.4	SpO ₂ Classification Report and Confusion Matrix	34
Fig.6.5	Blood Pressure Alert Prediction	35
Fig.6.6	Blood Pressure Classification Report and Confusion Matrix	35
Fig.6.7	Temperature Alert Prediction	36
Fig.6.8	Temperature Classification Report and Confusion Matrix	36
Fig.6.9	Performance Summary of Models	37
Fig.6.10	No Alert Output	38
Fig.6.11	Medium Alert Output	39
Fig.6.12	High Alert Output (Patient Dashboard)	40
Fig.6.13	High Alert Output	41
Fig.6.14	Final Model of IoT-Based Risk Prediction System using ML with Web Enabled Alerts	42
Fig.6.15	Patient Dashboard Output	42
Fig.6.16	Hospital Dashboard Output	43

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION TO HEALTH RISK PREDICTION SYSTEM

In modern healthcare systems, effective monitoring of patients after hospital discharge remains a critical challenge, particularly for individuals with chronic illnesses, post-surgical conditions, and elderly patients whose health status may change without immediate supervision. In the absence of proper monitoring, early signs of deterioration may go undetected, increasing the risk of complications and hospital readmissions. Conventional healthcare practices rely on scheduled check-ups and patient self-reporting, which are insufficient for capturing continuous variations in patient health conditions. Although predictive models have been developed to estimate hospital readmission risk, they are primarily limited to analysis and do not support timely intervention during critical situations.

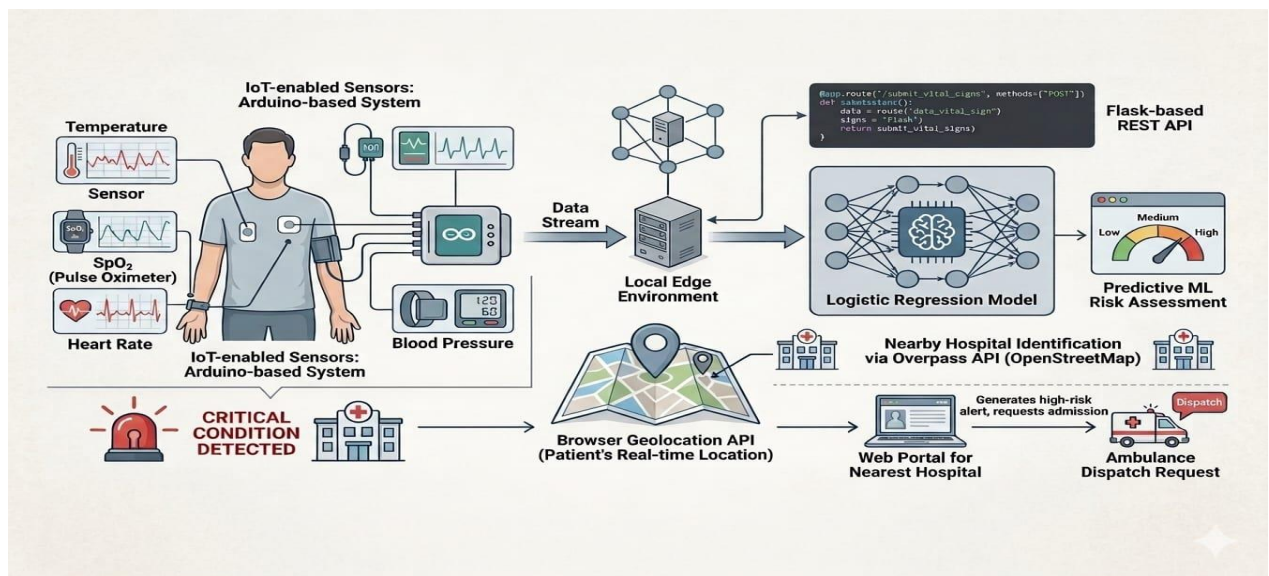


Fig.1.1 IoT-Based Health Risk Prediction System Using ML With Web Enabled Alerts

This creates a need for intelligent systems that can provide continuous supervision and detect health issues at an early stage. By integrating the Internet of Things (IoT) and Machine Learning (ML), patient data can be gathered through sensor-based devices and analyzed to track changes in health conditions over time. This supports early recognition of potential risks and enables healthcare providers to take timely action during critical situations, thereby improving patient safety. This project develops an IoT-based health risk prediction system that evaluates real-time patient data to determine current risk levels, with data collection, processing, and communication managed through a Flask-based backend ensuring seamless interaction between hardware components, the

prediction model, and the user interface. A web-based dashboard displays patient data and prediction results for continuous monitoring, while in critical situations, the system identifies the nearest hospital, generates emergency alerts, and supports ambulance response through the hospital interface. By integrating real-time monitoring, risk assessment, and emergency response, the system enhances patient safety, enables faster medical intervention, and contributes to more efficient and proactive healthcare management.

1.2 AIM

The aim of this project is to develop an IoT-based health risk prediction system that continuously monitors patient vital signs through sensor-based devices, analyzes real-time data using machine learning to assess health risk levels, and automatically sends alerts to the nearest hospital in critical situations, supporting timely medical intervention and the dispatch of ambulances to enhance patient safety and promote proactive healthcare management.

1.3 MOTIVATION

In modern healthcare, post-discharge patients often lack continuous monitoring due to limited resources and high patient load, increasing the risk of delayed complication detection. The IoT-based Health Risk Prediction System continuously tracks patient data in real time and uses machine learning to identify early health risks, while web-based alerts notify healthcare providers instantly, enabling timely intervention and improving patient safety.

1.4 PROBLEM STATEMENT

One of the major challenges in post-discharge patient care is effective monitoring, particularly for patients with chronic conditions and elderly individuals. The absence of continuous supervision makes it difficult to detect early signs of health deterioration. Conventional methods such as periodic check-ups and self-reporting fail to capture real-time health variations. This leads to delayed medical response during critical situations, resulting in increased hospital readmissions.

1.5 OBJECTIVE

To design and develop an IoT-based health risk prediction system integrated with machine learning that enables real-time patient vital monitoring, risk assessment, and patient location tracking with nearest hospital identification. The system aims to send automated alerts to the nearest hospital during critical health conditions to facilitate rapid ambulance dispatch and prompt clinical assistance, thereby improving patient outcomes.

1.6 APPLICATIONS

1. **Home-Based Monitoring:** Patients recovering at home, including elderly individuals living alone, can have their vitals continuously monitored in real time without visiting a hospital.
2. **Rural and Remote Healthcare:** Patients in remote areas far from hospitals can receive timely emergency response through automated hospital alerts and ambulance dispatch.
3. **Hospital Emergency Management:** Receiving real-time alerts and dispatching ambulances for incoming critical patients.
4. **Chronic Disease Management:** Monitoring patients with diabetes, hypertension, and cardiovascular conditions requiring continuous vital tracking.

1.7 ORGANISATION OF THESIS

Chapter 1 – Introduction covers the Aim, Motivation, Problem Statement, objective and Application of the proposed system.

Chapter 2 – Literature Survey presents a review of three reference papers related to IOT-based Health monitoring and machine learning in healthcare, concluding with key observations.

Chapter 3 – System Design and Methodology describes the proposed system Architecture, Block diagram, Flow Chart and the Methodology adopted for development.

Chapter 4 – Hardware Implementation explains the hardware components used and their circuit implementation.

Chapter 5 – Software Implementation covers the software tools used, machine learning model and Flask-based web application development.

Chapter 6 – Results and Analysis presents the model results, system output, and the final model of the system.

CHAPTER 2

LITERATURE SURVEY

Relevant journals and research papers on patient monitoring and health risk prediction have been reviewed and analyzed. IoT enables real-time collection of patient data, while machine learning techniques are used to identify potential health risks. This chapter highlights key studies, focusing on their methodologies, technologies used, and identified limitations.

2.1 REFERENCE PAPER 1

TITLE: Predictive Analytics for Hospital Readmissions Using Logistic Regression and IoT Sensor Data.

AUTHORS: Kiruthiga B, R. Sarasu, Arshadh Ariff Mohamed Abuthahir Riazulhameed, N. Mohankumar, Rajapriya M, A. Kasthuri

YEAR OF PUBLICATION: 2024

INTRODUCTION:

Hospital readmissions remain a critical and costly issue in modern healthcare systems, creating a significant burden on patients and medical institutions. Early identification of high-risk patients is essential for improving care and reducing unnecessary hospital visits. This paper presents a predictive analytics framework that integrates real-time IoT sensor data with a Logistic Regression model, using vital signs and historical records to generate risk assessments for better clinical management and healthcare efficiency.

THEME OF THE JOURNAL:

The primary focus of this paper is the development of an IoT-based predictive analytics system for hospital readmission, utilizing sensors to continuously monitor vital health parameters such as heart rate, blood pressure, and oxygen levels. The collected data is integrated with Electronic Health Records to form a complete health profile, which is then analyzed using a Logistic Regression model to predict readmission likelihood. The system enables early identification of potential health risks, supporting healthcare professionals in making timely decisions and improving overall patient management.

LIMITATIONS:

The system faces challenges related to data privacy, interoperability, and scalability, with performance heavily dependent on the accuracy and quality of the collected IoT sensor data

2.2 REFERENCE PAPER 2

TITLE: IoT-Enabled Advanced Health Monitoring System using ESP32 and UBIDOTS

AUTHORS: Balamanikandan A, Siddavatam Lakshmi Neeraja, Thathireddy Venkata Kishore, Venkatachalam K, Udatha Deepika, Settipalli Prathyusha.

YEAR OF PUBLICATION: 2024

INTRODUCTION

This paper presents a real-time health monitoring system using the ESP32 microcontroller to continuously measure and transmit vital health parameters. Traditional monitoring systems are largely confined to hospital settings, limiting continuous patient observation. To address this, the system enables remote and home-based patient monitoring using IoT and cloud connectivity, allowing healthcare providers to access patient data and support timely monitoring and improved care quality.

THEME OF THE JOURNAL:

The This paper focuses on developing a real-time, IoT-based wearable health monitoring system using the ESP32 microcontroller integrated with multiple sensors to collect patient vital parameters. The system displays readings locally through an LCD while triggering buzzer alerts for abnormal conditions. NODEMCU serves as a communication gateway, transmitting the collected data to the UBIDOTS cloud platform, enabling healthcare professionals and caretakers to remotely access and monitor patient vitals from any location, supporting early detection of critical health issues and improving overall patient care.

LIMITATIONS:

The system lacks intelligent risk classification, relies on a stable internet connection for data transmission, uses a basic buzzer-based alert mechanism, and has not been validated on a large-scale patient population, limiting its clinical applicability.

2.3 REFERENCE PAPER 3

TITLE: Smart Health Monitoring System Using Pulse and Temperature Sensor (Sensor-Based Health Observer)

AUTHORS: Mrunal Fatangare, Hemlata Ohal, Vilas Rathod

YEAR OF PUBLICATION: 2023

INTRODUCTION

This paper proposes a smart health observer system that utilizes biomedical sensors to monitor a patient's vital signs, specifically pulse rate and body temperature. The system is designed to alert concerned parties via SMS when abnormal health parameters are detected, facilitating timely medical attention.

THEME OF THE JOURNAL:

This paper focuses on developing a smart, sensor-based health monitoring system designed to provide continuous patient observation beyond hospital settings, particularly for rural and home-based environments. Biomedical sensors are interfaced with a Wi-Fi module controller to collect and display vital data on an LCD or serial monitor. Real-time health information is transmitted to concerned parties through SMS notifications, allowing family members to remotely access patient data with ease. The system aims to simplify home-based care for elderly individuals by enabling regular health checks and early detection of potential medical emergencies.

LIMITATIONS:

The system's reliance on SMS for alerts may introduce delays due to network issues. Additionally, the use of Wi-Fi modules necessitates a stable internet connection, which may not be available in all regions, potentially limiting the system's effectiveness.

2.4 CONCLUSION ON SURVEY PAPERS

The reviewed papers highlight the growing role of IoT and machine learning in real-time health monitoring. The existing systems primarily focus on predictive analysis and continuous monitoring of patient vitals. While these systems are capable of detecting abnormal health conditions, they are limited to generating basic alerts such as buzzers or SMS notifications and do not support any form of immediate medical response.

To address these limitations, the proposed system integrates IoT-based vital sign monitoring with machine learning models for accurate health risk prediction. In addition, the system tracks the patient's location, identifies the nearest hospital, and automatically sends an alert with patient details and location to the hospital's web portal, allowing hospital staff to review the case and trigger ambulance dispatch for immediate medical assistance.

CHAPTER 3

SYSTEM DESIGN AND METHODOLOGY

The system design and methodology of the proposed IoT-based health risk prediction system is presented in this chapter. It describes the overall architecture of the system, explaining how the hardware and software components are integrated to achieve real-time health monitoring and risk prediction. The proposed system overview, block diagram, and flowchart and methodology illustrate the complete working of the system from sensor data collection to emergency alert and ambulance dispatch.

3.1 PROPOSED SYSTEM

The proposed system is an IoT-based health risk prediction framework that continuously monitors patient vital parameters such as heart rate, SpO₂, temperature, and blood pressure, and predicts risk levels using machine learning. When a critical condition is detected, an emergency alert is automatically sent to the nearest hospital, where the hospital staff receives the alert and dispatches an ambulance for immediate medical assistance.

This system enables real-time health monitoring of patients, provides early detection of critical conditions, reduces delays in emergency response, and improves patient safety and survival rates. It also supports remote healthcare and smart medical systems while reducing overall healthcare costs by minimizing unnecessary hospital visits and ensuring timely intervention.

3.2 BLOCK DIAGRAM

The IoT-based health risk prediction system begins with the sensor module, which monitors body temperature using a DHT11 sensor, SpO₂ using a pulse oximeter, and blood pressure and heart rate using a blood pressure sensor.

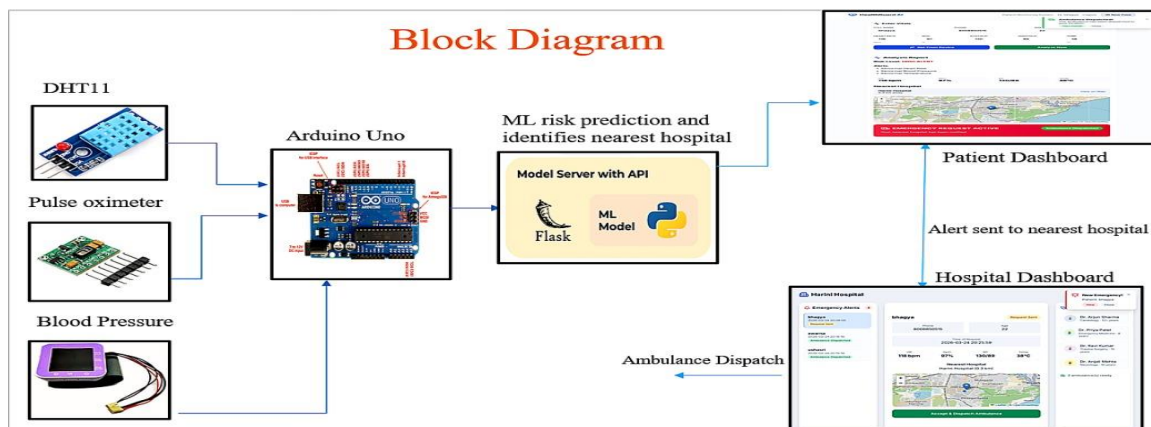


Fig.3.1 Block Diagram

The sensor data is collected by the Arduino Uno and transmitted to a Flask-based server, where a logistic regression models analyses the data to predict the patient's health risk. If a critical condition is detected, the system updates the patient dashboard with current readings and predicted risk level, identifies the nearest hospital, and sends an emergency alert with the patient's vital data and location. The hospital receives the alert and dispatches an ambulance to the patient's location, ensuring immediate medical assistance.

3.3 FLOW CHART AND METHODOLOGY:

The flowchart represents the complete working of the IoT-based health risk prediction system as shown in Fig 3.2 Patient vital signs such as Temperature, SpO₂, Heart Rate, and Blood Pressure are collected using sensors.

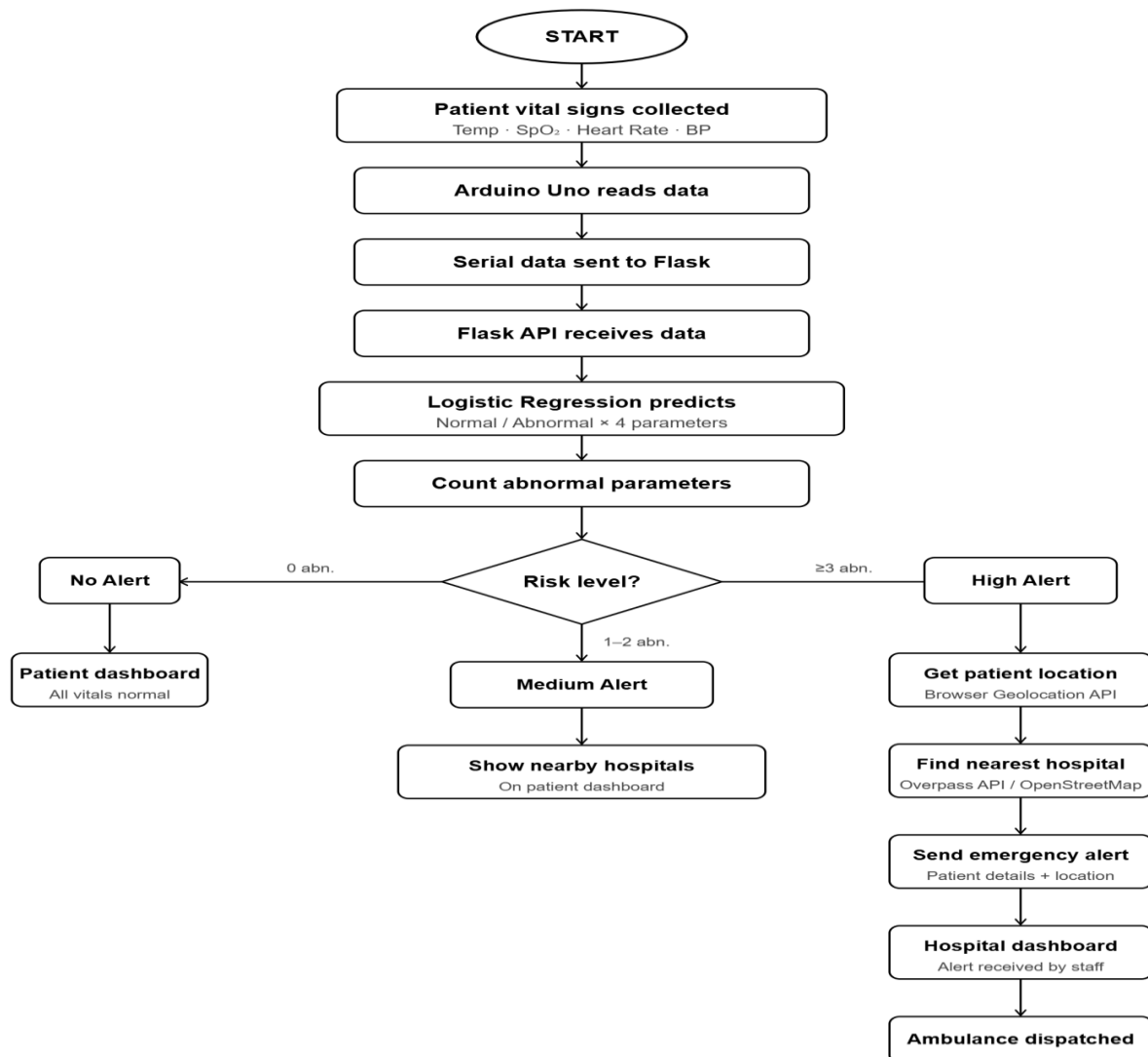


Fig.3.2 Flowchart

The Arduino Uno reads the sensor data and transmits it to the Flask server via serial communication. The Flask API receives and processes the incoming data. The Logistic Regression models classify each parameter as Normal (0) or Abnormal (1). The system then counts the number of abnormal parameters and determines the risk level. If all parameters are normal, a No Alert condition is triggered and the patient dashboard is updated with normal status. If 1 to 2 parameters are abnormal, a Medium Alert is raised and nearby hospitals are displayed on the patient dashboard. If 3 or more parameters are abnormal, a High Alert is triggered. In this case, the patient's location is obtained using the Browser Geolocation API, the nearest hospital is identified using the Overpass API, an emergency alert with patient details and location is sent to the hospital, and the hospital staff dispatches an ambulance for immediate medical care.

3.4 CONCLUSION

The system design of the IoT-based health risk prediction system was presented in this chapter. The proposed system architecture integrates hardware sensors, machine learning models, Flask backend, and web applications to achieve real-time patient health monitoring and automated emergency response. The block diagram illustrates the complete hardware and software integration, while the flowchart describes the step-by-step working of the system from vital data collection to ambulance dispatch. The design ensures continuous patient monitoring, accurate risk classification, and timely alert generation. Based on the system architecture, the hardware components and circuit connections are implemented and presented in the Hardware Implementation section.

CHAPTER 4

HARDWARE IMPLEMENTATION

An IOT-Based health monitoring system is designed to continuously monitor key patient vital parameters such as body temperature, heart rate, oxygen saturation (SpO₂), and blood pressure. An Arduino-based microcontroller serves as the core processing unit, interfacing with various sensors to collect patient data accurately in real time. The collected data is then transmitted to a local edge computing system for further processing. The hardware implementation ensures reliable data acquisition, low power consumption, and portability, making it suitable for continuous and remote patient monitoring.

4.1 COMPONENTS AND TOOLS REQUIRED

1. Arduino Microcontroller
2. DHT11 Temperature Sensor
3. MAX30102 Pulse Oximeter Sensor
4. Blood Pressure Sensor (BP + HR)
5. Arduino IDE

4.1.1 ARDUINO UNO

The Arduino Uno is the central processing unit of the system, built around the ATmega328P microcontroller operating at 5V with a clock speed of 16 MHz. It consists of 14 digital I/O pins and 6 analog input pins.

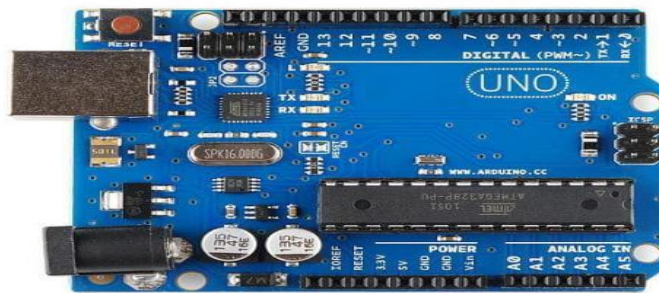


Fig.4.1 Arduino Uno

In this system, the Arduino Uno reads data from all connected sensors. The Temperature Sensor is connected to Digital Pin 3, the MAX30102 Pulse Oximeter is connected via I2C communication using SDA (A4) and SCL (A5) pins, and the Blood Pressure Sensor is connected via serial communication using TX (Pin 1) and RX (Pin 2) pins. The collected vital parameters are transmitted to the Flask backend via serial communication for further processing.

4.1.2 TEMPERATURE SENSOR

The DHT11 is a digital temperature sensor used to measure body temperature in the system. It provides calibrated digital output and operates within a voltage range of 3.3V to 5V, making it suitable for interfacing with Arduino.

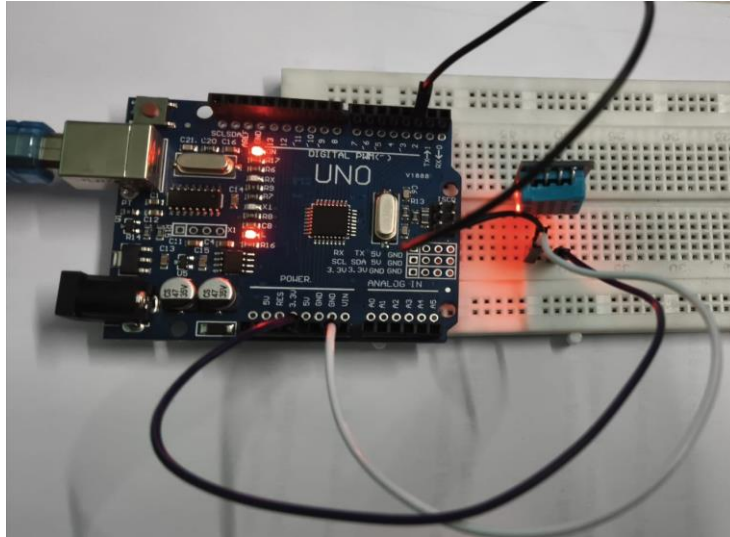


Fig.4.2 Temperature Sensor

The sensor has four pins VCC, Data, NC, and GND where VCC is connected to 5V, GND to ground, and the Data pin is connected to Digital Pin 2 of the Arduino. It measures temperature in the range of 0°C to 50°C with an accuracy of $\pm 2^{\circ}\text{C}$ and provides readings at a rate of one reading per second. In this system, it continuously monitors patient body temperature and transmits the data to the Arduino for further processing.

4.1.3 MAX30100 PULSE OXIMETER

The MAX30100 is a pulse oximeter sensor designed for non-invasive measurement of blood oxygen saturation (SpO_2). It works by shining infrared and red light through the finger. When the heart pumps blood, the amount of light passing through the finger changes.

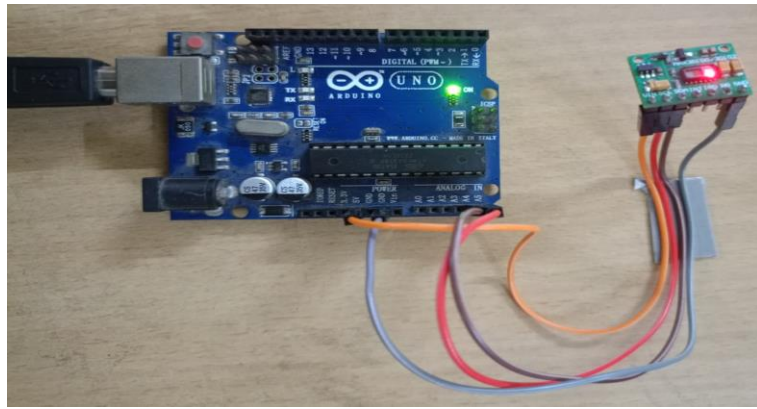


Fig 4.3 Pulse Oximeter Sensor

The sensor detects these changes and calculates the oxygen level in the blood based on how much light is absorbed. It communicates with the Arduino via the I2C protocol using four pins VCC, GND, SDA, and SCL where VCC is connected to 3.3V, GND to ground, SDA to pin A4, and SCL to pin A5. It is designed for portable devices, consuming very low power while delivering accurate and reliable real-time measurements. In this system, it continuously monitors the patient's SpO₂ levels and transmits the data to the Arduino for further processing.

4.1.4 BLOOD PRESSURE SENSOR

The Blood Pressure Sensor is a wrist-type digital sensor used to measure the patient's systolic blood pressure (SYS), diastolic blood pressure (DIA), and heart rate (PUL). It works on the oscillometric method, where the cuff inflates and applies pressure on the wrist artery. As the cuff deflates, it detects oscillations in the artery wall caused by blood flow and calculates the blood pressure and heart rate values.



Fig. 4.4 Blood pressure sensor

The measured values are displayed on the built-in LCD screen and transmitted to the Arduino via serial communication using four pins VCC, GND, TX, and RX where VCC is connected to 5V, GND to ground, TX to Pin 2, and RX to Pin 3 of the Arduino. In this system, it continuously monitors the patient's blood pressure and heart rate values and transmits the data to the Arduino for further processing.

4.1.5 ARDUINO IDE

The Arduino Integrated Development Environment (IDE) is a software tool used to write, compile, and upload code to the Arduino Uno microcontroller. It supports the C and C++ programming languages and provides a simple and user-friendly interface for programming the Arduino board.

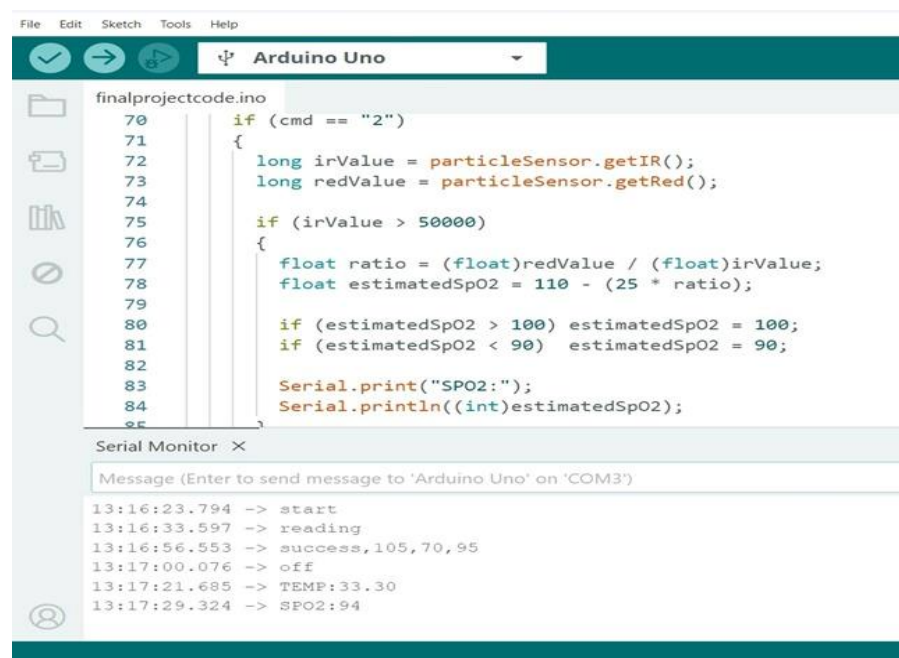


Fig.4.5 Arduino IDE

The Arduino Integrated Development Environment (IDE) is a software tool used to write, compile, and upload code to the Arduino Uno microcontroller. It supports the C and C++ programming languages and provides a simple and user-friendly interface for programming the Arduino board. In this system, the Arduino IDE is used to write the sensor interfacing code, configure the serial communication settings, and upload the program to the Arduino Uno to enable continuous reading and transmission of patient vital data to the Flask backend.

4.2 CIRCUIT IMPLEMENTATION

The circuit is designed to connect multiple sensors to the Arduino UNO to monitor a patient's vital signs in real time. The **DHT11 temperature sensor** is powered through the Arduino's 5V and GND pins, with its data line connected to digital pin 3 for transmitting temperature readings.

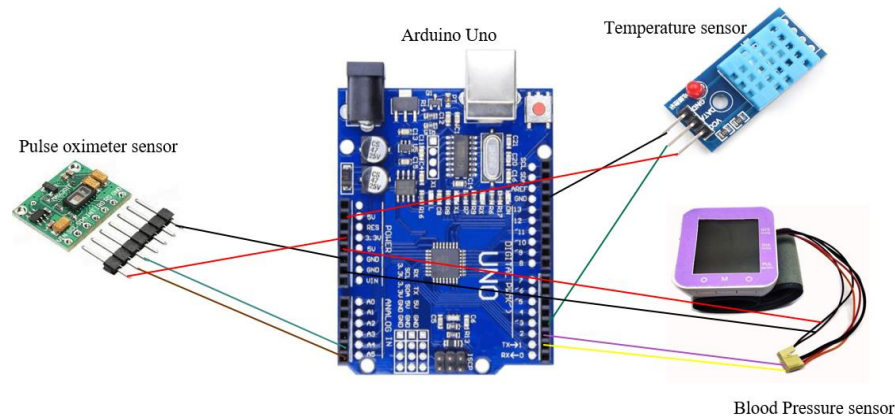


Fig.4.6 Circuit Implementation

The **pulse oximeter sensor** communicates via the I2C protocol, with VCC and GND connected to 5V and GND, and the SCL and SDA lines linked to pins A5 and A4. The **digital blood pressure monitor** sends systolic, diastolic, and pulse measurements through serial communication, with VCC and GND connected to 5V and GND, TX to digital pin 1 (RX), and RX to digital pin 2 (TX). All sensors are powered through the Arduino, which collects and processes the data to provide a comprehensive, real-time display of vital parameters.

4.3 HARDWARE OUTPUT

The hardware setup of the IoT-based health risk prediction system is shown in the figure above. The DHT11 temperature sensor, MAX30100 pulse oximeter sensor, and digital blood pressure sensor are successfully interfaced with the Arduino Uno.

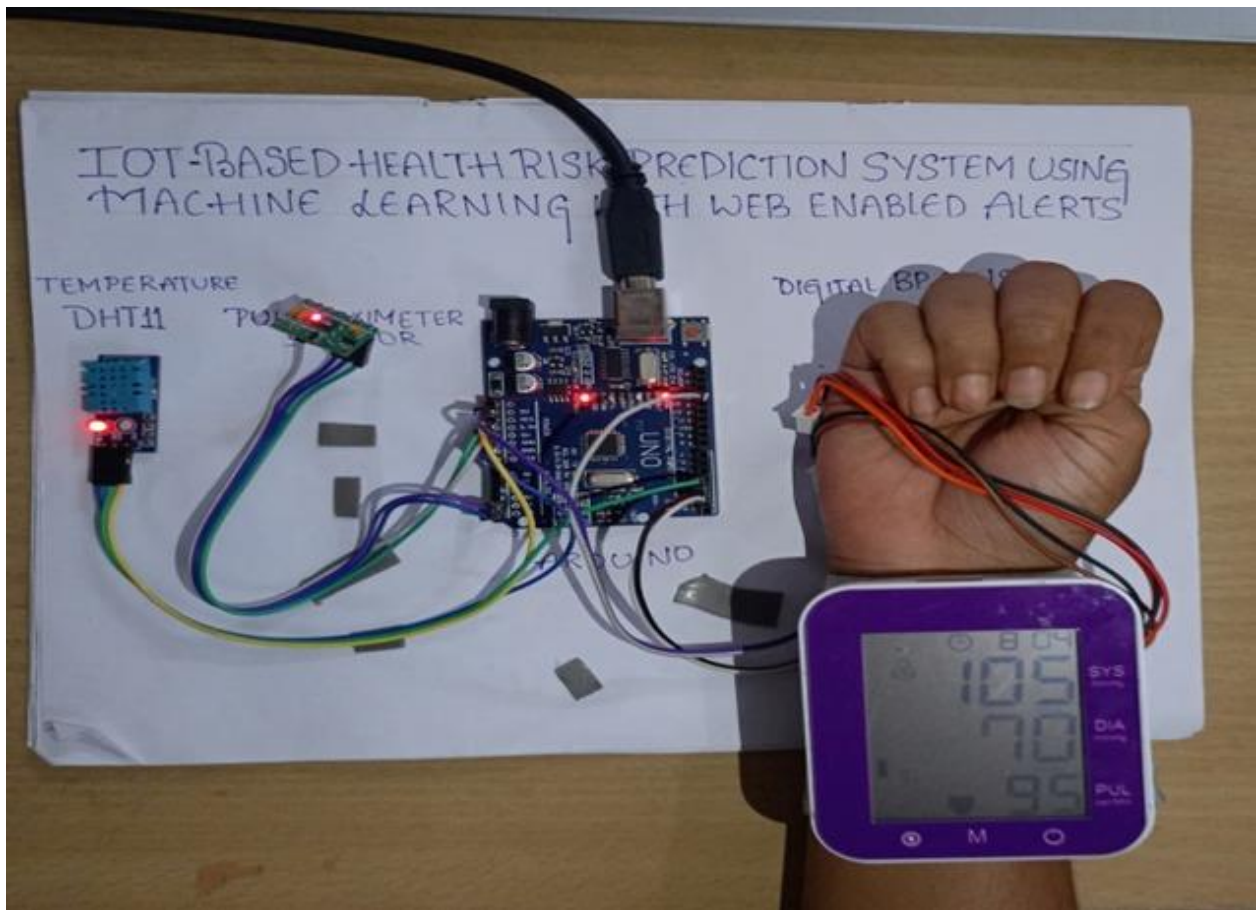


Fig.4.7 Hardware Output

The blood pressure sensor is worn on the wrist to measure systolic blood pressure, diastolic blood pressure, and heart rate. The DHT11 sensor measures the patient's body temperature and the MAX30100 sensor measures the SpO₂ level. All sensors are connected to the Arduino Uno, which continuously reads and transmits the real-time patient vital data to the Flask backend via serial communication.

4.4 CONCLUSION

The hardware setup provides a reliable framework for real-time monitoring of patient vital parameters. The microcontroller efficiently handles data collection, ensuring precise and consistent readings. All components work together to maintain energy efficiency and operational stability. The design is portable and practical for continuous monitoring in various environments. Overall, the implementation delivers a robust and dependable platform for health monitoring applications. This establishes a foundation for software implementation, where the collected data is processed, analyzed, and utilized for risk prediction and emergency alert generation.

CHAPTER 5

SOFTWARE IMPLEMENTATION

The software implementation of the IoT-based health risk prediction system is presented in this chapter. The implementation begins with the software tools used, followed by the development of machine learning models using Python in Google Colab with libraries such as NumPy, Pandas, Scikit-learn, and Joblib. The trained models are deployed into the Flask backend, which receives real-time patient data from the Arduino via Pyserial and classifies the patient risk level. The Browser Geolocation API and Overpass API are used to identify the nearest hospital and send emergency alerts for high-risk cases. The patient and hospital web applications are developed using HTML, CSS, and JavaScript with Flask templates.

5.1 SOFTWARE REQUIRMENTS

1. Python 3.1.4
2. Google Colab
3. Flask
4. Scikit-learn, NumPy and Pandas
5. Browser Geolocation API
6. Overpass API / OpenStreetMap
7. HTML, CSS, JavaScript

5.1.1 PROGRAMMING ENVIRONMENT

Python is the primary programming language used for developing the system. The machine learning models are developed and trained in **Google Colab**. **NumPy** and **Pandas** are used for data handling and preprocessing of the patient dataset. **Scikit-learn** is used for implementing and training the **Logistic Regression** models. **Joblib** is used for saving the trained models as .pkl files for deployment.

5.1.2 BACKEND FRAMEWORK

Flask is used to build the backend server that manages data transmission from the Arduino, loads the trained models for real-time prediction, classifies the patient risk level, and serves the results to the web applications.

5.1.3 SERIAL COMMUNICATION

PySerial is used to establish serial communication between the Arduino and the Flask backend, enabling real-time transmission of patient vital data to the system.

5.1.4 WEB INTERFACE

HTML, **CSS**, and **JavaScript** are used along with **Flask templates** to develop the patient and hospital web applications, providing a secure and real-time interface for health monitoring and emergency response.

5.1.5 LOCATION SERVICES

The **Browser Geolocation API** is used to obtain the patient's real-time location when a high-risk condition is detected. The **Overpass API** from OpenStreetMap is then used to identify the nearest hospital based on the patient's location and send emergency alerts.

5.2 LOGISTIC REGRESSION MODEL DEVELOPMENT

5.2.1 INTRODUCTION

Logistic Regression is used in the proposed system to classify each patient vital parameter such as Heart Rate, SpO₂, Blood Pressure, and Body Temperature as Normal (0) or Abnormal (1). A dataset of 50,000 patient records collected from Kaggle is used to train the models. Logistic Regression is selected due to its simplicity, low computational cost, and ability to produce clear and interpretable predictions for binary classification problems, while supporting real-time integration with the Flask backend. Four independent models are trained, one for each vital parameter, and saved as .pkl files for deployment into the Flask application

5.2.2 FLOWCHART

The flowchart illustrates the complete process of Logistic Regression model development, from dataset collection to model saving.

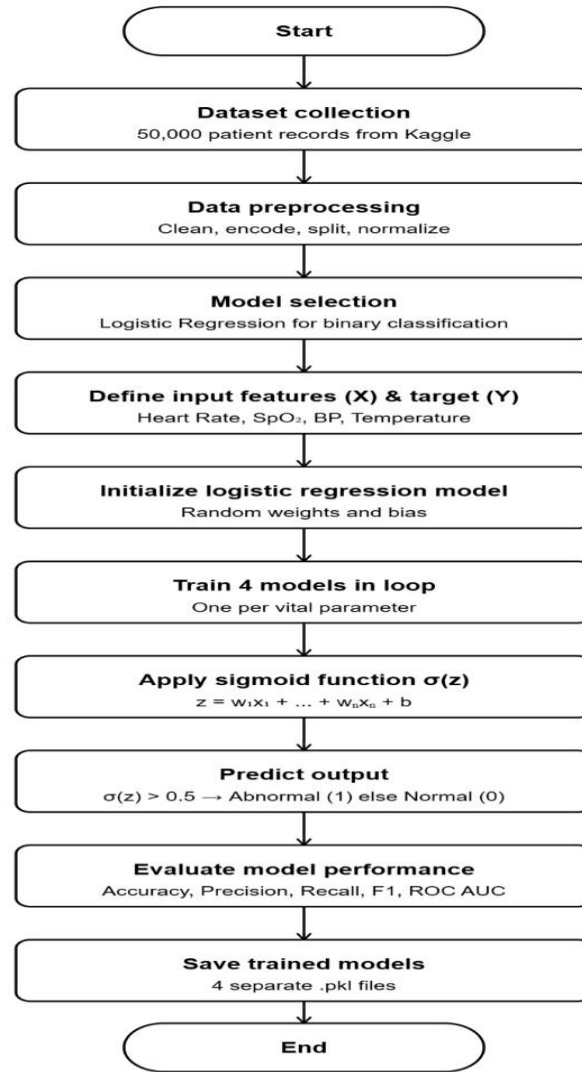


Fig.5.1 Flowchart

5.2.3 DATASET COLLECTION

The dataset used for training the machine learning models was collected from the Kaggle website. It consists of 50,000 patient records, where each record contains four vital parameters: Heart Rate, SpO₂ (Oxygen Saturation), Blood Pressure, and Body Temperature. Each parameter is individually labeled as Normal or Abnormal based on standard medical thresholds. Since the health conditions of each vital sign vary independently, separate models are trained for each parameter to ensure accurate and reliable classification.

Patient Number	Heart Rate (bpm)	SpO2 Level (%)	Blood Pressure (mmHg)	Body Temperature (°C)	Heart Rate Alert	SpO2 Level Alert	Blood Pressure Alert	Temperature Alert
1	96	92	101	89	36.90468046	Normal	Normal	Normal
2	76	83	103	85	37.25412936	Normal	Low	Abnormal
3	92	88	123	70	36.53941754	Normal	Low	Normal
4	137	84	167	97	37.01876654	High	Low	Normal
5	76	81	175	80	37.32867133	Normal	Low	Abnormal
"	"	"	"	"	"	"	"	"
"	"	"	"	"	"	"	"	"
49994	68	97	168	90	36.78387065	Normal	Normal	High
49995	144	80	168	75	37.97602996	High	Low	High
49996	118	96	103	65	37.1129653	High	Normal	Normal
49997	61	81	101	71	36.45750742	Normal	Low	Normal
49998	63	91	166	67	37.08383178	Normal	Normal	High
49999	95	85	140	64	37.8272786	Normal	Low	Normal
50000	77	94	169	97	36.70418259	Normal	Normal	High

Fig.5.2 Dataset

5.2.4 DATA PREPROCESSING

Data preprocessing is a crucial step performed before training the machine learning models to ensure the quality, consistency, and reliability of the dataset. The following steps were carried out:

1. **Data Cleaning:** Null values, duplicates, and outliers were removed
2. **Feature Selection:** Heart Rate, SpO₂, Blood Pressure, and Body Temperature were selected as input features.
3. **Label Encoding:** Labels were encoded as Normal (0) and Abnormal (1).
4. **Train-Test Split:** Dataset was divided into 80% training and 20% testing data
5. **Normalization:** Input features were scaled to a uniform range for fair and accurate classification.

5.2.5 MODEL SELECTION

Model selection is a critical step in developing an effective machine learning system. After analysing the nature of the problem and the dataset, **Logistic Regression** was identified as the most suitable algorithm for this system due to the following reasons:

1. **Binary Classification:** Well-suited for Normal (0) and Abnormal (1) classification.
2. **Interpretability:** Produces clear predictions with probability scores.

3. **Low Computational Cost:** Requires minimal computational resources, ideal for real-time deployment.
4. **Easy Integration:** Integrates seamlessly with the Flask-based REST API.

5.2.6 MODEL TRAINING

The Logistic Regression models are developed and trained in Python using Google Colab. The models are trained on the preprocessed patient dataset to learn the relationship between vital parameters and their corresponding risk labels. The training process is carried out in the following steps:

1. The pre-processed dataset is provided as input with Heart Rate, SpO₂, Blood Pressure, and Body Temperature as input features.
2. Logistic Regression models are initialized with random weights and bias.
3. Four separate models are trained, each for a specific vital parameter — Heart Rate, SpO₂, Blood Pressure, and Body Temperature Classification Models.
4. A loop is used to train each model, where the same input features are used but the target label corresponds to one specific parameter at each iteration.
5. For each model, the weighted sum is computed:

$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

$$z = \mathbf{w} \mathbf{i} \mathbf{x} \mathbf{i} + \mathbf{b}$$

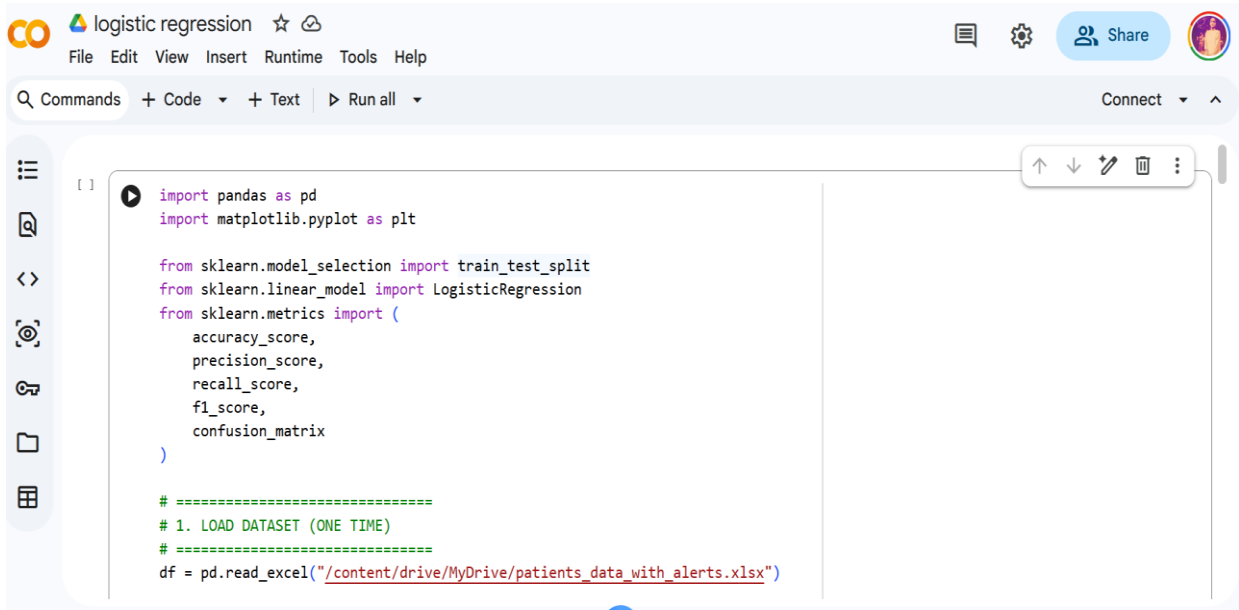
where x_i are features, w_i are weights, and b is bias.

6. The value z is passed through the Sigmoid Function to produce an output between 0 and 1:

$$7. \sigma(z) = \frac{1}{1+e^{-z}}$$

8. The output is classified as Abnormal (1) if $\sigma(z) > 0.5$, otherwise Normal (0).
9. The predicted output is compared with the actual labels to compute the error.

10. Weights and bias are adjusted to minimize the error using optimization techniques.
11. The process repeats until the model achieves optimal performance.
12. All four trained models are saved separately as .pkl files — Heart Rate, SpO₂, Blood Pressure, and Temperature Models.



The screenshot shows a Google Colab notebook interface. The title bar says 'logistic regression'. The menu bar includes 'File', 'Edit', 'View', 'Insert', 'Runtime', 'Tools', and 'Help'. Below the menu bar, there are tabs for 'Commands', 'Code', 'Text', and 'Run all'. The 'Code' tab is active, showing a Python script. The script imports pandas as pd and matplotlib.pyplot as plt. It then imports train_test_split from sklearn.model_selection, LogisticRegression from sklearn.linear_model, and accuracy_score, precision_score, recall_score, f1_score, and confusion_matrix from sklearn.metrics. A comment indicates the next step is to load the dataset. The code then reads an Excel file from the path '/content/drive/MyDrive/patients_data_with_alerts.xlsx' into a DataFrame named df.

```
[ ]  
import pandas as pd  
import matplotlib.pyplot as plt  
  
from sklearn.model_selection import train_test_split  
from sklearn.linear_model import LogisticRegression  
from sklearn.metrics import (  
    accuracy_score,  
    precision_score,  
    recall_score,  
    f1_score,  
    confusion_matrix  
)  
  
# =====  
# 1. LOAD DATASET (ONE TIME)  
# =====  
df = pd.read_excel("/content/drive/MyDrive/patients_data_with_alerts.xlsx")
```

Fig.5.3 Model Training in Google Colab

5.2.7 MODEL EVALUATION & PERFORMANCE

After training, each model was tested on the 20% test data to check how well it classifies the health parameters. The following metrics were used to measure the performance:

- Train Accuracy: How correctly the model predicted on the training data.
- Test Accuracy: How correctly the model predicted on the new test data.
- Precision: Out of all predicted abnormal cases, how many were actually abnormal.
- Recall: Out of all actual abnormal cases, how many were correctly identified.
- F1-Score: Overall balance between Precision and Recall.
- ROC AUC: How well the model separates Normal and Abnormal cases.

The machine learning models were successfully trained, evaluated, and saved as .pkl files. These models form the intelligent core of the system and are integrated into the Flask backend for real-time patient risk prediction.

5.3 FLASK BACKEND & WEB APPLICATION

5.3.1 INTRODUCTION TO FLASK

Flask is a lightweight Python web framework used to build web servers, APIs, and applications. In this system, Flask acts as the central backend server that connects the hardware sensors, machine learning models, and the web applications. It manages the flow of data between all components of the system. Flask uses routes to link backend functions with web pages and supports external APIs to find nearby hospitals. It is simple, fast, flexible, and easy to maintain, making it suitable for real-time healthcare applications.

5.3.2 FLASK APPLICATION SETUP

The Flask application was set up by following the steps below:

Step 1 – Environment Setup : Python was installed and a virtual environment was created and activated using `myenv\Scripts\activate` to manage project dependencies separately.

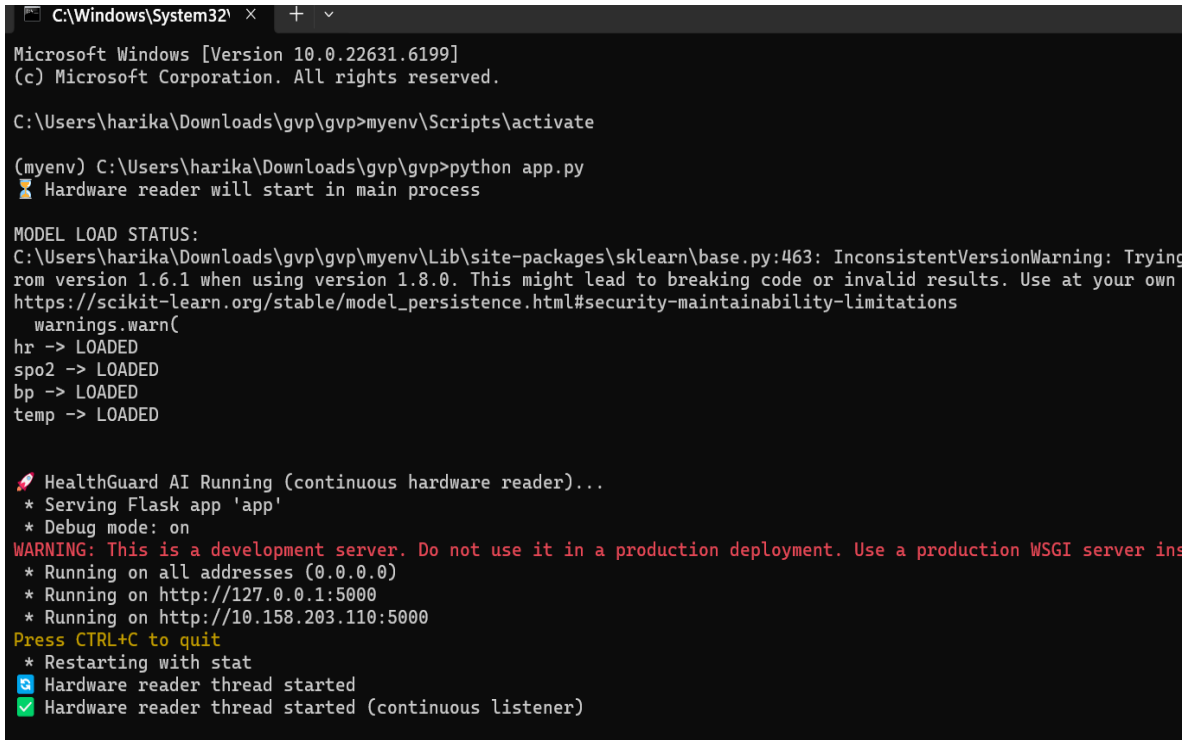
Step 2 – Flask Installation : Flask was installed in the virtual environment using `pip install flask`, providing the framework for backend development and handling HTTP requests.

Step 3 – Project Folder Structure : The project folder was organized as follows:

- `app.py` — Main Flask application file containing routes and backend logic.
- `Templates/` — Stores HTML pages for patient and hospital interfaces.
- `static/` — Contains CSS and JavaScript for styling and functionality.
- `models/` — Includes trained .pkl model files for predictions.
- This structure ensures modularity and clear separation of components.

Step 4 – Application Initialization: The Flask application was initialized in app.py by creating an application instance to manage routing, request handling, and integration with machine learning models and serial data.

Step 5 – Running the Application: The application was executed using python app.py, which starts the server, enables communication with Arduino, processes incoming data, and displays results on the web interface.



```
C:\Windows\System32 x + v
Microsoft Windows [Version 10.0.22631.6199]
(c) Microsoft Corporation. All rights reserved.

C:\Users\harika\Downloads\gvp\gvp>myenv\Scripts\activate

(myenv) C:\Users\harika\Downloads\gvp\gvp>python app.py
Hardware reader will start in main process

MODEL LOAD STATUS:
C:\Users\harika\Downloads\gvp\gvp\myenv\Lib\site-packages\sklearn\base.py:463: InconsistentVersionWarning: Trying
rom version 1.6.1 when using version 1.8.0. This might lead to breaking code or invalid results. Use at your own
https://scikit-learn.org/stable/model_persistence.html#security-maintainability-limitations
  warnings.warn(
hr -> LOADED
spo2 -> LOADED
bp -> LOADED
temp -> LOADED

HealthGuard AI Running (continuous hardware reader)...
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server ins
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://10.158.203.110:5000
Press CTRL+C to quit
* Restarting with stat
Hardware reader thread started
Hardware reader thread started (continuous listener)
```

Fig.5.4 Flask Application Setup

5.3.3 DATA TRANSMISSION FROM ARDUINO TO FLASK

The Arduino continuously collects real-time patient vital data: Heart Rate, SpO₂, Blood Pressure, and Body Temperature from the connected sensors and transmits it to the Flask backend via serial communication. The Flask application reads the incoming serial data using Python's pyserial library. The received data is then extracted and formatted into the required input structure and sent to the machine learning models for prediction.

```

10.229.137.110 - - [22/Feb/2026 14:53:03] "GET /get_latest_hardware HTTP/1.1" 200 -
10.229.137.110 - - [22/Feb/2026 14:53:04] "GET /login HTTP/1.1" 200 -
10.229.137.110 - - [22/Feb/2026 14:53:09] "POST /login HTTP/1.1" 302 -
10.229.137.110 - - [22/Feb/2026 14:53:09] "GET / HTTP/1.1" 200 -
10.229.137.110 - - [22/Feb/2026 14:53:09] "GET /get_latest_hardware HTTP/1.1" 200 -
10.229.137.110 - - [22/Feb/2026 14:53:11] "GET /get_latest_hardware HTTP/1.1" 200 -
RAW: 'success,111,77,99'
10.229.137.110 - - [22/Feb/2026 14:53:13] "GET /get_latest_hardware HTTP/1.1" 200 -
RAW: 'off'
❌ Measurement done → requesting temp & spo2
RAW (temp): 'TEMP:31.30'
10.229.137.110 - - [22/Feb/2026 14:53:16] "GET /get_latest_hardware HTTP/1.1" 200 -
RAW (spo2): 'SPO2:93'
✅ New hardware reading stored
10.229.137.110 - - [22/Feb/2026 14:53:18] "GET /get_latest_hardware HTTP/1.1" 200 -
10.229.137.110 - - [22/Feb/2026 14:53:20] "GET /get_latest_hardware HTTP/1.1" 200 -
10.229.137.110 - - [22/Feb/2026 14:53:23] "GET /get_latest_hardware HTTP/1.1" 200 -
10.229.137.110 - - [22/Feb/2026 14:53:25] "GET /get_latest_hardware HTTP/1.1" 200 -
10.229.137.110 - - [22/Feb/2026 14:53:26] "GET /hospital HTTP/1.1" 200 -
10.229.137.110 - - [22/Feb/2026 14:53:27] "GET /get_latest_hardware HTTP/1.1" 200 -
10.229.137.110 - - [22/Feb/2026 14:53:28] "GET /get_latest_hardware HTTP/1.1" 200 -
DEBUG: Found 0 hospitals near (17.6774, 83.2036)

```

Fig.5.5 Data Transmission to Flask

5.3.4 MODEL DEPLOYMENT INTO FLASK

The Logistic Regression models are trained in Google Colab and saved as **.pkl files**. These saved files contain the learned weights and parameters of each trained model. The .pkl files are loaded into the Flask application (app.py) at runtime. When real-time sensor data is received from Arduino, it is passed to the loaded models for risk prediction.

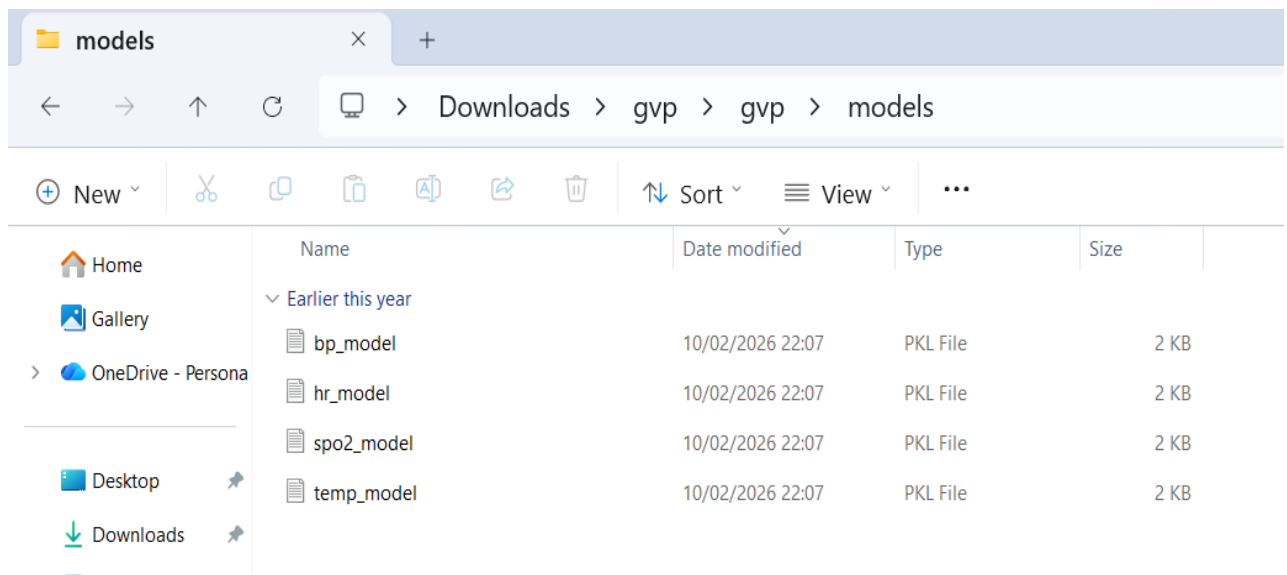


Fig.5.6 Models Deployment into Flask

5.3.5 PATIENT WEB APPLICATION

The patient web application was developed using **HTML**, **CSS**, **JavaScript**, and **Flask** templates. It provides a secure interface for patient families to monitor their patient's health status in real time. The application includes the following pages:

- **Registration Page:** Allows patient to create an account by entering their details.
- **Login Page:** Provides secure access to the dashboard using registered credentials.
- **Patient Dashboard:** Displays the patient's real-time sensor values, machine learning prediction results, nearest hospital information, and emergency alert status. All patient readings along with the predicted risk level are automatically stored in a CSV file for records and future analysis.

The image displays two side-by-side web forms. The left form is titled 'Login' and contains two input fields labeled 'Username' and 'Password'. Below these fields is a blue button labeled 'Login'. At the bottom, there is a link that says 'Don't have an account? Register'. The right form is titled 'Register' and contains four input fields labeled 'Username', 'Password', 'Full Name', and 'Phone', followed by an 'Age' field. Below these fields is a blue button labeled 'Register'. At the bottom, there is a link that says 'Already have an account? Login'.

Fig.5.7 Login Page and Registration

 Enter Vitals

FULL NAME

PHONE

AGE

e.g., name

+91 9xxxxxxx

25

HEART RATE

SPO₂

SYSTOLIC

DIASTOLIC

TEMP

72

98

120

80

36.6

bpm

%

°C

Get From Device

Analyze Now

Fig.5.8 Patient Dashboard

timestamp	hr	spo2	sys	dia	temp	risk
24/03/2026 19:57	70	98	115	77	36	NO ALERT
24/03/2026 19:58	71	99	115	76	36.5	NO ALERT
24/03/2026 19:58	100	99	115	76	36.5	NO ALERT
24/03/2026 19:58	100	99	117	80	36.5	NO ALERT
24/03/2026 19:59	80	97	117	74	36.5	NO ALERT
24/03/2026 19:59	69	100	121	79	35.5	NO ALERT
24/03/2026 20:00	120	98	120	79	36	MEDIUM ALERT
24/03/2026 20:00	99	78	121	80	36	MEDIUM ALERT
24/03/2026 20:01	82	99	110	76	36	NO ALERT
24/03/2026 20:01	134	98	113	79	36	MEDIUM ALERT
24/03/2026 20:03	130	90	122	80	36.5	MEDIUM ALERT
24/03/2026 20:06	124	99	120	80	38	MEDIUM ALERT
24/03/2026 20:18	103	87	129	86	38	HIGH ALERT

Fig.5.9 Data Storage

5.3.6 HOSPITAL WEB APPLICATION

The hospital web application was developed using HTML, CSS, JavaScript, and Flask templates. It provides a secure platform for hospital staff to manage and respond to patient emergency alerts. The application includes the following pages:

- **Registration Page:** Allows hospital staff to create an account by entering their details.
- **Login Page:** Provides secure access to the hospital dashboard using registered credentials.
- **Hospital Dashboard:** Displays patient emergency alerts, details, and location, and allows staff to dispatch an ambulance for immediate medical response.

Hospital Login

Username

Password

Login

New hospital? [Register here](#)

Hospital Registration

Username

Password

Hospital Name

Address

Register Hospital

Already have an account? [Login](#)

Fig.5.10 Hospital Login Page and Registration Page

Harini Hospital

OnlineLogout

Emergency Alerts1

Kandrapu Nikitha

2026-03-08 19:27:28

Ambulance Dispatched

Kandrapu Nikitha

Ambulance Dispatched

Phone6305938614Age21

Time of Request2026-03-08 19:27:25

HR123 bpm

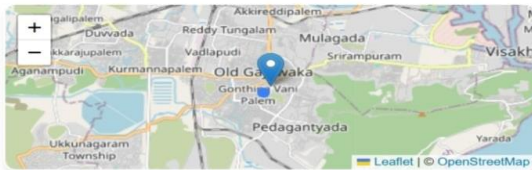
SpO₂87%

BP125/90

Temp38.1°C

Nearest Hospital

Harini Hospital (0.3 km)



Accept & Dispatch Ambulance

On Duty Staff

Dr. Arjun Sharma

Cardiology · 12+ years

Dr. Priya Patel

Emergency Medicine · 8 years

Dr. Ravi Kumar

Trauma Surgery · 15 years

Dr. Anjali Mehta

Neurology · 10 years

3 ambulance(s) ready

Fig.5.11 Hospital Dashboard

28

5.3.7 FLASK SERVER INTEGRATION

The Flask server acts as the central hub that connects all components of the system hardware sensors, machine learning models, and web applications enabling them to work together seamlessly. The complete system flow is as follows:

1. The Python environment is activated and the Flask application (app.py) is started, enabling the Flask backend and serial communication.
2. Flask loads all four trained Logistic Regression models into memory.
3. Real-time patient vital data is received from Arduino via serial communication.
4. The received data is passed to the loaded models for Normal (0) or Abnormal (1) prediction.
5. Predictions from all four models are received and the patient risk level is determined by counting abnormal predictions:

0 abnormal → No Alert

1–2 abnormal → Medium Alert

≥3 abnormal → High Alert

6. The patient dashboard is updated in real-time with the prediction results.
7. For high-risk cases, the patient's location is obtained through the **Browser Geolocation API**. The **Overpass API** from OpenStreetMap is then used to identify nearby hospitals based on the patient's coordinates.
8. An emergency alert with patient details and location is sent to the nearest hospital's web portal.
9. The hospital staff reviews the alert and triggers ambulance dispatch from the hospital dashboard.


HealthGuard AI

Patient Monitoring System
Hi, K.Nikitha
Logout
New Case

Enter Vitals

FULL NAME

PHONE

AGE

Kandrapu Nikitha

6305938614

21

HEART RATE

SPO₂

SYSTOLIC

DIASTOLIC

TEMP

96

94

139

87

35.6

bpm

%

°C

Get From Device

Analyze Now

Analysis Report

Risk Level: MEDIUM ALERT

Alerts:

- Abnormal Blood Pressure

HR

96 bpm

SPO₂

94%

BP

139/87

TEMP

35.6°C

Nearby Hospitals

Harini Hospital

0.3 km away

View on Map

Gowtami Hospital

0.6 km away

View on Map

Visakha Child Care Centre

0.8 km away

View on Map

Ankitha Hospital

0.8 km away

View on Map

R K Hospital

0.9 km away

View on Map

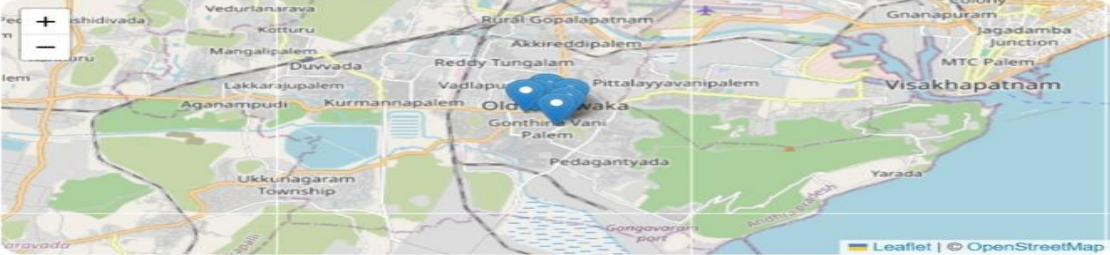


Fig.5.12 Patient Risk Analysis

5.3.8 CONCLUSION

The software implementation of the IoT-based health risk prediction system was successfully completed. Logistic Regression models were trained on 50,000 patient records, each vital parameter is classified as Normal or Abnormal. The trained models are deployed into the Flask backend, which manages data transmission from the Arduino, risk level classification, and emergency alert generation. For high-risk cases, the system identifies the nearest hospital and sends an emergency alert, allowing the hospital staff to dispatch an ambulance for immediate medical response. The patient and hospital web applications provide a secure and real-time interface for health monitoring and emergency response, with all patient data stored for future analysis. The performance of the trained models and the overall system output are presented in the Results and Analysis.

CHAPTER 6

RESULTS AND ANALYSIS

The IoT-based health risk prediction system was evaluated to assess its performance and functionality. The evaluation is carried out in two parts — the machine learning model performance and system output. The machine learning models are assessed based on their classification accuracy and performance metrics. The system output validates the end-to-end functionality by testing the patient under different health conditions and verifying the corresponding alert generation and emergency response.

6.1 LOGISTIC REGRESSION MODEL RESULTS

The trained Logistic Regression models were evaluated on the 20% test dataset to measure their ability to correctly classify each vital parameter as Normal or Abnormal. Each model was assessed using Accuracy, Precision, Recall, F1-Score, and ROC AUC metrics. A confusion matrix was generated for each model to visualize the correctly and incorrectly classified instances. The evaluation results of each model are presented in the following sections.

6.1.1 HEART RATE MODEL

The Heart Rate classification model was trained to classify heart rate values as Normal (0) or Abnormal (1). The model performance is as follows:

Train Accuracy: 100% | **Test Accuracy:** 100%

Precision: 100% | **Recall:** 100% | **F1-Score:** 100% | **ROC AUC:** 100%

The **confusion matrix** shows that out of the total test instances, 4595 instances were correctly classified as Normal and 5405 instances were correctly classified as Abnormal, with zero misclassification. The model perfectly distinguishes between normal and abnormal heart rate conditions.

```

tmp/ipykernel_727/3082064779.py:127: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a future version. To retain
y_hr = df["Heart Rate Alert"].replace(binary_map)
tmp/ipykernel_727/3082064779.py:138: FutureWarning: Downcasting behavior in `replace` is deprecated and will be removed in a future version. To retain
"Actual Binary":sample_hr["Heart Rate Alert"].replace(binary_map),

```

Heart Rate Alert Prediction

Heart Rate (bpm)	Actual	Predicted	Actual Binary	Predicted Binary
91	Normal	Normal	0	0
75	Normal	Normal	0	0
135	Abnormal	Abnormal	1	1
78	Normal	Normal	0	0
141	Abnormal	Abnormal	1	1
85	Normal	Normal	0	0
66	Normal	Normal	0	0
65	Normal	Normal	0	0
114	Abnormal	Abnormal	1	1
138	Abnormal	Abnormal	1	1

Fig.6.1 Heart Rate Alert Prediction

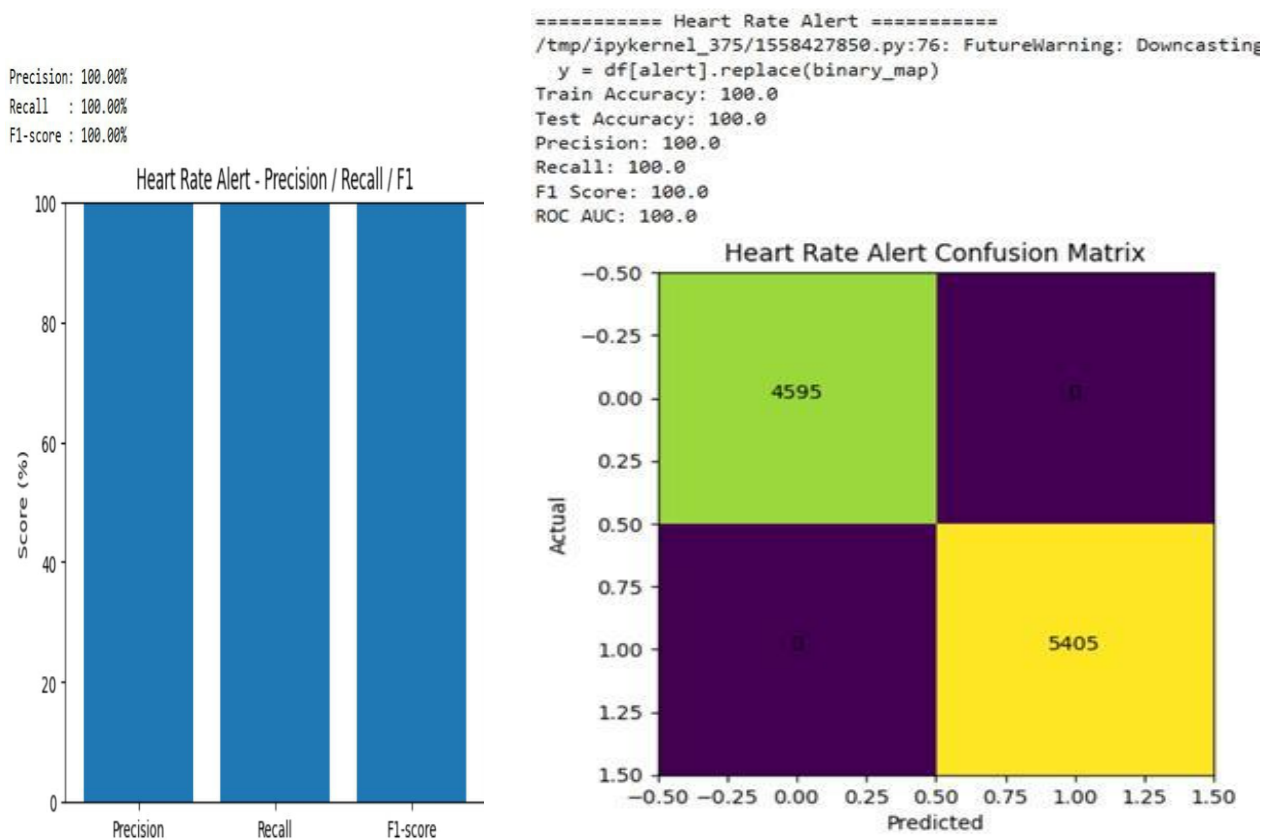


Fig.6.2 Heart Rate Classification Report and Confusion Matrix

6.1.2 SPO₂ MODEL

The SpO₂ classification model was trained to classify oxygen saturation levels as Normal (0) or Abnormal (1). The model performance is as follows:

Train Accuracy: 100% | Test Accuracy: 100%

Precision: 100% | Recall: 100% | F1-Score: 100% | ROC AUC: 100%

The **confusion matrix** shows that out of the total test instances, **5006** instances were correctly classified as Abnormal and **4994** instances were correctly classified as Normal, with zero misclassification. The model perfectly distinguishes between normal and abnormal SpO₂ conditions.

```
/tmp/ipykernel_727/3082064779.py:151: FutureWarning: Downcasting behavior in 'replace' is deprecated and will be removed in a future version. To retain
y_spo2 = df["SpO2 Level Alert"].replace(binary_map)
/tmp/ipykernel_727/3082064779.py:162: FutureWarning: Downcasting behavior in 'replace' is deprecated and will be removed in a future version. To retain
"Actual Binary":sample_spo2["SpO2 Level Alert"].replace(binary_map),
```

SpO2 Level (%)	Actual	Predicted	Actual Binary	Predicted Binary
82	Abnormal	Abnormal	1	1
88	Abnormal	Abnormal	1	1
95	Normal	Normal	0	0
94	Normal	Normal	0	0
93	Normal	Normal	0	0
84	Abnormal	Abnormal	1	1

Fig.6.3 SpO₂ Alert Prediction

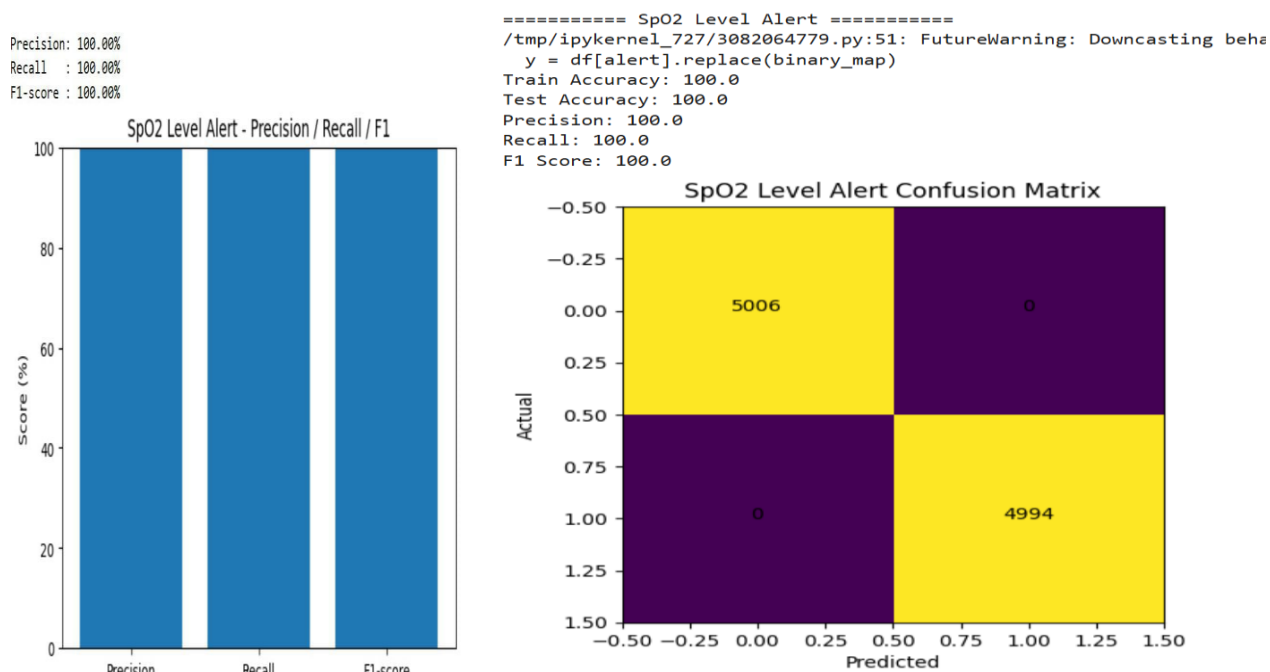


Fig.6.4 Spo2 Classification report and Confusion Matrix

6.1.3 Blood Pressure Model

The Blood Pressure classification model was trained to classify blood pressure values as Normal (0) or Abnormal (1). The model performance is as follows:

Train Accuracy: 86.97% | **Test Accuracy:** 87.17%

Precision: 89.35% | **Recall:** 89.19% | **F1-Score:** 89.27% | **ROC AUC:** 95.35%

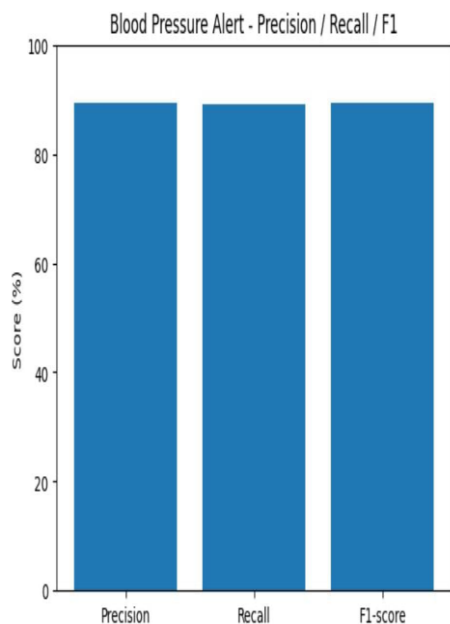
The **confusion matrix** shows that **3379** instances were correctly classified as Normal and **5338** as Abnormal. However, **636** Normal instances were misclassified as Abnormal and **647** Abnormal instances were misclassified as Normal due to overlapping boundary values in the dataset.

```
/tmp/ipykernel_727/3082064779.py:181: FutureWarning: Downcasting behavior in 'replace' is deprecated and will be removed in a future version. To retain the current behavior, use 'y_bp = df["Blood Pressure Alert"].replace(binary_map, dtype=bool)'
y_bp = df["Blood Pressure Alert"].replace(binary_map)
/tmp/ipykernel_727/3082064779.py:199: FutureWarning: Downcasting behavior in 'replace' is deprecated and will be removed in a future version. To retain the current behavior, use 'sample_bp["Blood Pressure Alert"] = sample_bp["Blood Pressure Alert"].replace(binary_map, dtype=bool)'
"Actual Binary":sample_bp["Blood Pressure Alert"].replace(binary_map),
```

Blood Pressure Alert Prediction

Systolic BP	Diastolic BP	Actual	Predicted	Actual Binary	Predicted Binary
171	66	Abnormal	Abnormal	1	1
160	96	Abnormal	Abnormal	1	1
133	70	Normal	Normal	0	0
158	93	Abnormal	Abnormal	1	1
124	97	Abnormal	Abnormal	1	1

Fig.6.5 Blood Pressure Alert Prediction



```
===== Blood Pressure Alert =====
/tmp/ipykernel_727/3082064779.py:51: FutureWarning: Downcasting behavior in 'replace' is deprecated and will be removed in a future version. To retain the current behavior, use 'y = df[alert].replace(binary_map, dtype=bool)'
y = df[alert].replace(binary_map)
Train Accuracy: 86.97
Test Accuracy: 87.17
Precision: 89.35
Recall: 89.19
F1 Score: 89.27
```

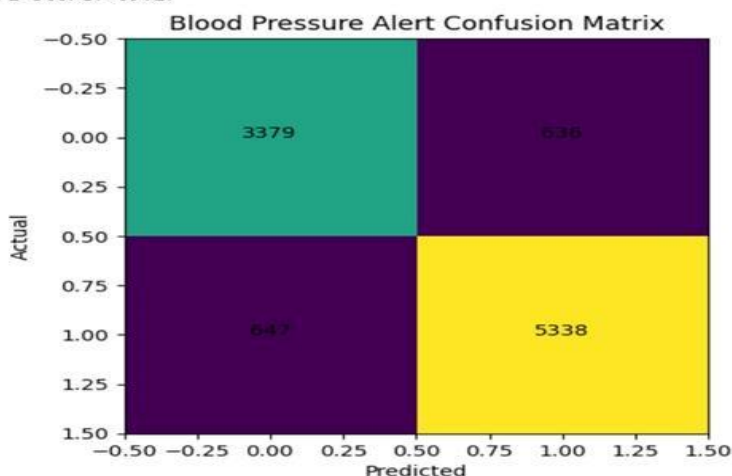


Fig.6.6 Blood Pressure Classification Report and Confusion Matrix

6.1.4 TEMPERATURE MODEL

The Body Temperature classification model was trained to classify temperature values as Normal (0) or Abnormal (1). The model performance is as follows:

Train Accuracy: 90.28% | Test Accuracy: 90.63%

Precision: 89.77% | Recall: 89.06% | F1-Score: 89.41% | ROC AUC: 89.06%

The confusion matrix shows that **5106** instances were correctly classified as Normal and **3957** as Abnormal. However, **451** Normal instances were misclassified as Abnormal and **486** Abnormal instances were misclassified as Normal.

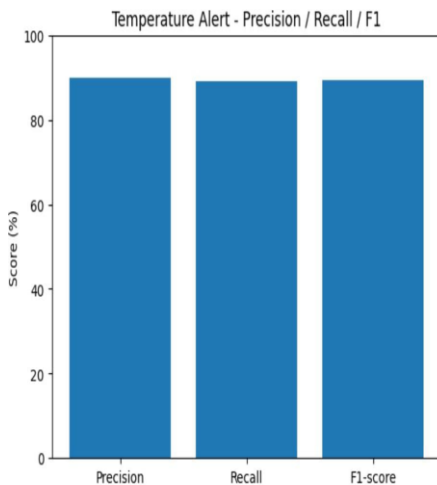
```
/tmp/ipykernel_727/3082064779.py:212: FutureWarning: Downcasting behavior in 'replace' is deprecated and will be removed in a future version. To ret
y_temp = df["Temperature Alert"].replace(binary_map)
/tmp/ipykernel_727/3082064779.py:223: FutureWarning: Downcasting behavior in 'replace' is deprecated and will be removed in a future version. To ret
"Actual Binary":sample_temp["Temperature Alert"].replace(binary_map),
```

Temperature Alert Prediction

Body Temperature (°C)	Actual	Predicted	Actual Binary	Predicted Binary
36.89991133019022	Normal	Normal	0	0
37.90938947747119	Abnormal	Abnormal	1	1
37.31244587058406	Abnormal	Abnormal	1	1
36.86933321029316	Normal	Normal	0	0
37.75248752915287	Abnormal	Abnormal	1	1

Fig.6.7 Temperature Alert Prediction

```
recision: 89.77%
recall : 89.06%
l-score : 89.41%
```



```
===== Temperature Alert =====
/tmp/ipykernel_727/3082064779.py:51: FutureWarning: Downcasting b
y = df[alert].replace(binary_map)
Train Accuracy: 90.28
Test Accuracy: 90.63
Precision: 89.77
Recall: 89.06
F1 Score: 89.41
```

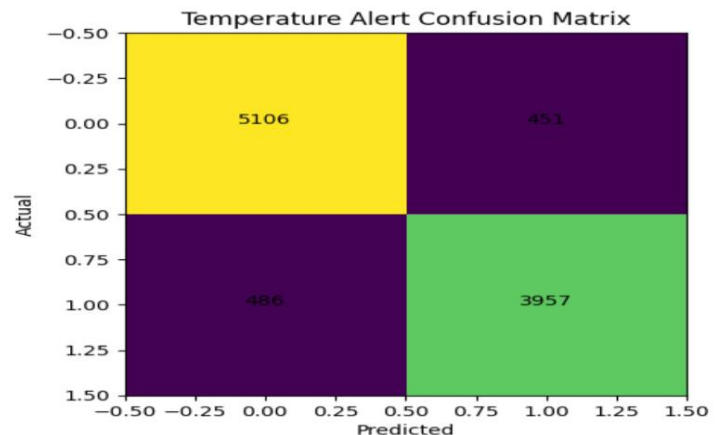


Fig. 6.8 Temperature Classification Report and Confusion Matrix

6.1.5 PERFORMANCE SUMMARY OF MODELS

Model	Train Accuracy (%)	Test Accuracy (%)	Precision (%)	Recall (%)	F1 Score (%)	ROC AUC (%)
Heart Rate Alert	100.0	100.0	100.0	100.0	100.0	100.0
SpO2 Level Alert	100.0	100.0	100.0	100.0	100.0	100.0
Blood Pressure Alert	86.97	87.17	89.35	89.19	89.27	95.35
Temperature Alert	90.28	90.63	89.77	89.06	89.41	89.06

Fig. 6.9 Performance Summary of Models

Overall, all four Logistic Regression models demonstrated strong and reliable classification performance. The Heart Rate and SpO₂ models achieved perfect accuracy. The Blood Pressure and Temperature models achieved good performance with minor misclassifications, as the values of these parameters have a very narrow difference between normal and abnormal ranges making it difficult for the model to correctly classify borderline cases. The results confirm that the models are well-suited for real-time patient health risk prediction.

6.2 SYSTEM OUTPUT

The trained Logistic Regression models independently classify each vital parameter as Normal (0) or Abnormal (1). The overall patient risk level is determined based on the total count of abnormal predictions, resulting in three distinct output conditions — No Alert, Medium Alert, and High Alert. Each condition triggers a specific system response, which is reflected on the patient and hospital dashboards. The output results for each condition are presented in the following sections.

6.2.1 NO ALERT CONDITION

When all four vital parameters are classified as Normal by the trained models, the system generates no alert. The patient dashboard displays the patient vital values and the risk level as “NO ALERT”, indicating that the patient’s health condition is stable and no medical intervention is required.

 **HealthGuard AI**

Patient Monitoring System Hi, K.Nikitha Logout [+ New Case](#)

 Enter Vitals

FULL NAME

PHONE

AGE

Kandrapu Nikitha

6305938614

21

HEART RATE

SPO₂

SYSTOLIC

DIASTOLIC

TEMP

99

93

111

77

31.3

bpm

%

°C

 Get From Device

Analyze Now

 **Analysis Report**

Risk Level: NO ALERT

No critical alerts detected

HR

99 bpm

SPO₂

93%

BP

111/77

TEMP

31.3°C

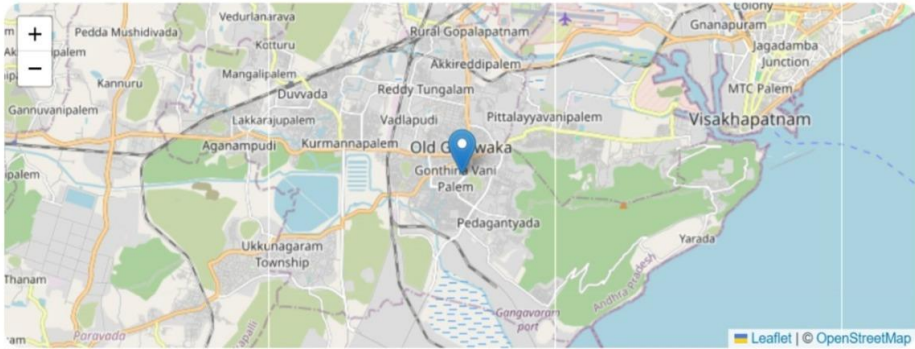


Fig 6.10 No Alert Output

6.2.2 MEDIUM ALERT CONDITION

When one or two vital parameters are classified as Abnormal by the trained models, the system generates a Medium Alert. The patient dashboard displays the patient vital values and the risk level as “**MEDIUM ALERT**” along with the abnormal parameter detected. The nearby hospitals are displayed on the dashboard with their distance from the patient’s location, allowing the patient's family to take quick action if the condition worsens.

The screenshot displays the HealthGuard AI Patient Monitoring System interface. At the top, the header includes the logo, the text "Patient Monitoring System", the user name "Hi, K.Nikitha", a "Logout" button, and a "New Case" button. The main section is titled "Enter Vitals" and contains input fields for patient information and vital signs. The patient's full name is "Kandrapu Nikitha", phone number is "6305938614", and age is "21". The vital signs entered are: Heart Rate (96 bpm), SPO₂ (94%), Systolic Blood Pressure (139), Diastolic Blood Pressure (87), and Temperature (35.6 °C). Below the input fields are two buttons: "Get From Device" and "Analyze Now".

The "Analysis Report" section shows a "Risk Level: MEDIUM ALERT" in orange text. Under "Alerts:", there is a list item "Abnormal Blood Pressure". Below this, four boxes display the vital signs: HR 96 bpm, SPO₂ 94%, BP 139/87, and TEMP 35.6°C.

The "Nearby Hospitals" section lists five hospitals with their distances from the patient's location:

- Harini Hospital (0.3 km away) - View on Map
- Gowtami Hospital (0.6 km away) - View on Map
- Visakha Child Care Centre (0.8 km away) - View on Map
- Ankitha Hospital (0.8 km away) - View on Map
- R K Hospital (0.9 km away) - View on Map

At the bottom, there is a map showing the patient's location in Visakhapatnam, India, with a blue dot indicating the patient's position. The map includes labels for various areas like Old Vengal Rao, Gonthira Vani, and Pedagantyada. The map is powered by Leaflet and OpenStreetMap.

Fig.6.11 Medium Alert Output

6.2.3 HIGH ALERT CONDITION

When three or more vital parameters are classified as abnormal, the system generates a High Alert. The patient dashboard clearly displays the vital values, marks the risk level as “HIGH ALERT,” and highlights all abnormal parameters. The system then identifies the nearest hospital and automatically sends an emergency alert with the patient’s details and location. The hospital receives the alert through its dashboard and immediately dispatches an ambulance to the patient location. All patient data is stored in XML format for record keeping and future analysis.

The screenshot displays the HealthGuard AI Patient Monitoring System interface. At the top, the header includes the logo, user name 'Hi, swarna', and links for 'Logout' and 'New Case'. The main section is titled 'Enter Vitals' and contains input fields for patient information: Full Name (swarna), Phone (8008850515), and Age (21). Below these are fields for vital signs: Heart Rate (103 bpm), SPO₂ (87%), Systolic (129), Diastolic (86), and Temp (38 °C). Two buttons, 'Get From Device' and 'Analyze Now', are at the bottom of this section.

The 'Analysis Report' section shows a 'Risk Level: HIGH ALERT' in red. It lists alerts: Abnormal Heart Rate, Low Oxygen Saturation, Abnormal Blood Pressure, and Abnormal Temperature. Below the alerts, a summary of vital signs is shown: HR 103 bpm, SPO₂ 87%, BP 129/86, and TEMP 38°C.

The 'Nearest Hospital' section identifies 'Harini Hospital' as being 0.3 km away, with a 'View on Map' link. A map of the area is displayed below, showing the patient's location relative to the hospital and surrounding landmarks.

A notification box in the top right corner states 'Ambulance Dispatched!' with the message 'The ambulance has been dispatched to your location.' and links for 'View Details' and 'Close'.

At the bottom, a red banner indicates 'EMERGENCY REQUEST ACTIVE' and 'Your nearest hospital has been notified', with a green button labeled 'Ambulance Dispatched'.

Fig.6.12 High Alert Output(Patient Dashboard)

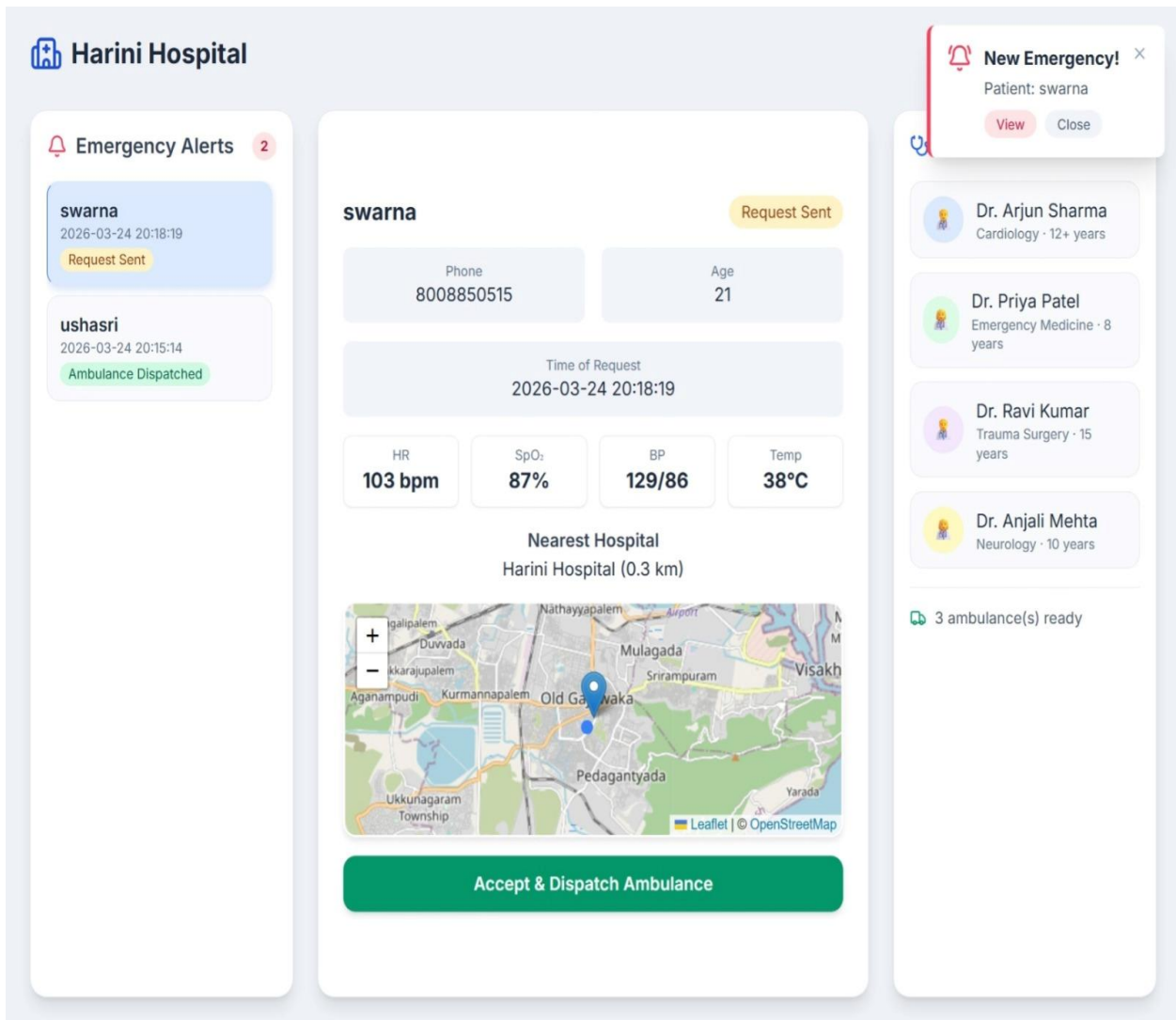


Fig.6.13 High Alert Output

6.3 FINAL MODEL

This figure shows the complete system workflow. The patient monitoring module collects vital parameters and sends the data to the Flask server for machine learning-based risk prediction. The patient dashboard displays the vital values and risk level. When a high-risk condition is detected, an alert is sent to the nearest hospital. The hospital dashboard receives the alert and enables ambulance dispatch. All patient data is stored for future analysis.

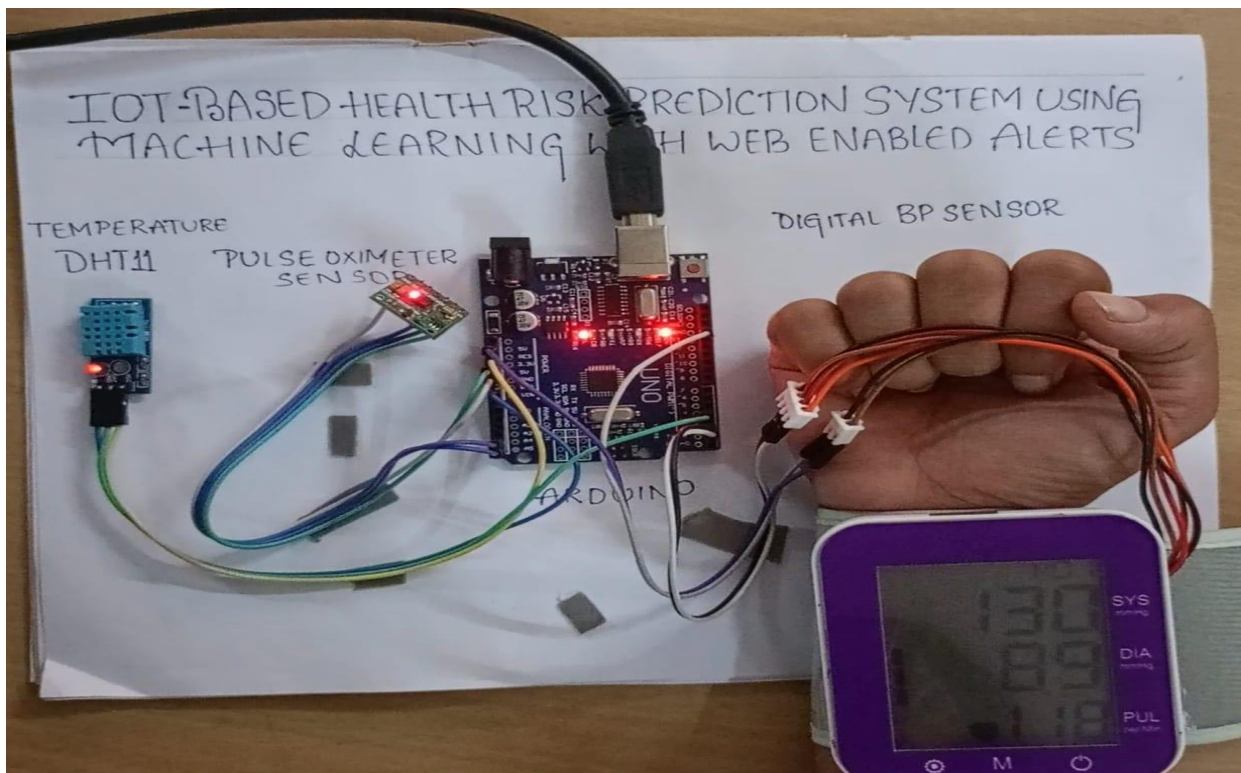


Fig.6.14 Final Model Of IOT-Based Risk Prediction System Using ML With Enabled Alerts

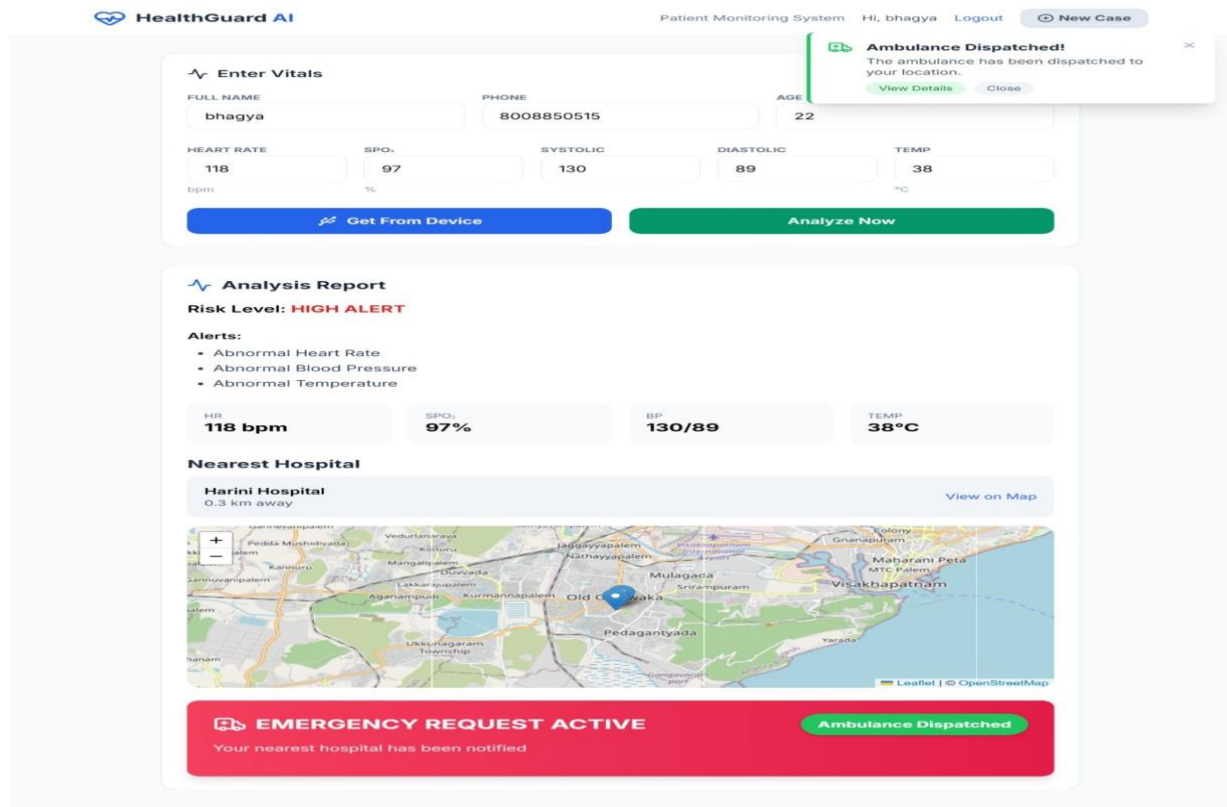


Fig 6.15 Patient Dashboard Output

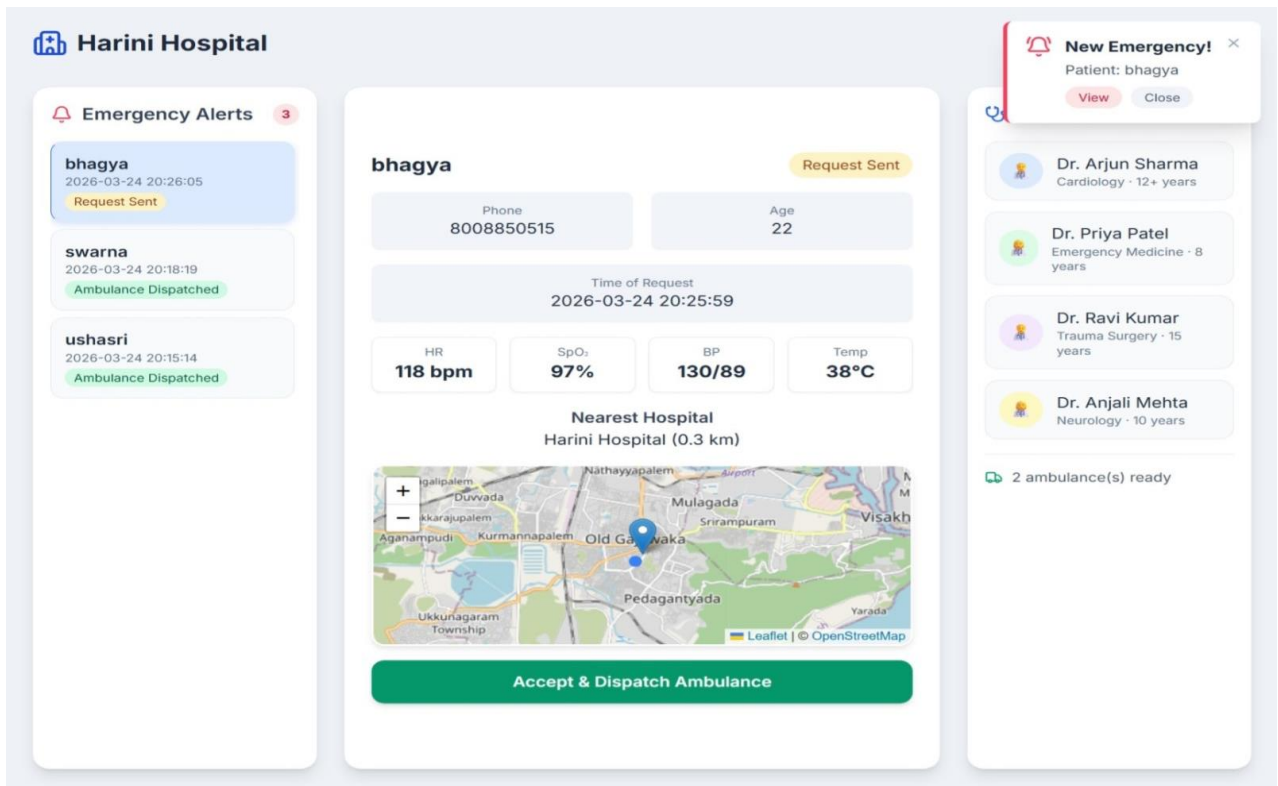


Fig 6.16 Hospital Dashboard Output (High Alert & Ambulance Dispatch)

6.4 CONCLUSION

The IoT-based health risk prediction system using machine learning with web-enabled alerts was successfully designed and implemented. The system integrates Arduino-based sensor monitoring, trained Logistic Regression models, a Flask backend, and patient and hospital web applications into a single unified framework. The system continuously monitors patient vital parameters and automatically classifies each parameter as Normal or Abnormal. Based on the count of abnormal predictions, the system generates No Alert, Medium Alert, or High Alert conditions. For high-risk cases, the system identifies the nearest hospital, sends an emergency alert with patient details and location, and enables ambulance dispatch for immediate medical response.

The machine learning models demonstrated strong classification performance, with the Heart Rate and SpO₂ models achieving 100% accuracy and the Blood Pressure and Temperature models achieving 87.17% and 90.63% accuracy respectively. The system successfully enables continuous patient monitoring, real-time risk prediction, and automated emergency alert generation, supporting timely medical intervention and improving overall patient safety.

6.5 FUTURE SCOPE

Although the proposed system achieves its intended objectives, several enhancements can be incorporated in the future to improve its capabilities:

1. **Advanced Machine Learning Models:** Deep learning algorithms can be explored to improve prediction accuracy, particularly for parameters with overlapping boundary values such as Blood Pressure and Temperature.
2. **Cloud Integration:** The system can be integrated with cloud platforms for remote data storage, access, and analysis.
3. **Mobile Application:** A dedicated mobile application can be developed for patients and families to monitor health status and receive alerts directly on their smartphones.
4. **Wearable Device Integration:** The sensor modules can be incorporated into wearable devices for continuous and comfortable patient monitoring.
5. **Multi-Patient Monitoring:** The system can be scaled to support simultaneous monitoring of multiple patients, making it suitable for large-scale healthcare applications.

6.6 REFERENCES

1. K. B, A. A. Mohamed Abuthahir Riazulhameed, R. M, R. Sarasu, N. Mohankumar and A. Kasthuri, "Predictive Analytics for Hospital Readmissions Using Logistic Regression and IoT Sensor Data," 2024 6th International Conference on Energy, Power and Environment (ICEPE).
2. B. A, S. L. Neeraja, T. V. Kishore, U. Deepika, S. Prathyusha and V. K, "IoT-Enabled Advanced Health Monitoring System using ESP32 and UBI DOTS," 2024 International Conference on IoT Based Control Networks and Intelligent Systems (ICICNIS).
3. M. Fatangare, H. Ohal and V. Rathod, "Smart Health Monitoring System Using Pulse and Temperature Sensor (Sensor-Based Health Observer)," 2023 7th International Conference on Computing, Communication, Control and Automation (ICCUBEA), Pune, India
4. Aosong Electronics Co., Ltd., "DHT11 Temperature and Humidity Sensor Datasheet," Guangzhou, China, 2010.
5. Maxim Integrated, "MAX30100 Pulse Oximeter and Heart-Rate Sensor IC for Wearable Health — Datasheet," San Jose, CA, USA, 2014.
6. Arduino Software: <https://www.arduino.cc/en/software>
7. **Coursera Flask course:** <https://www.coursera.org/learn/python-project-for-ai-application-development>
8. **Machine Learning:** <https://developers.google.com/machine-learning/crash-course/logistic-regression>